

Міністерство освіти і науки України
Національний університет “Львівська політехніка”



Кафедра ЕОМ

**ДОСЛІДЖЕННЯ ІНТЕГРОВАНОГО НА КРИСТАЛІ МІКРОКОНТРОЛЕРА
АНАЛОГО-ЦИФРОВОГО ПЕРЕТВОРЮВАЧА**

Методичні вказівки

до лабораторної роботи № 1 з дисципліни “Мікропроцесорні системи”
для студентів спеціальності 6.123.00.00 - “Комп’ютерна інженерія”

Затверджено
на засіданні кафедри
”Електронні обчислювальні машини”
Протокол № 1 від 28. 08. 2020 року

Львів – 2020

Дослідження інтегрованого на кристалі мікроконтролера аналого-цифрового перетворювача: Методичні вказівки до лабораторної роботи № 1 з курсу “Мікропроцесорні системи” для студентів спеціальності 6.123.00.00 - “Комп’ютерна інженерія” / Укладач: Пуйда В.Я.,– Львів: Національний університет “Львівська політехніка”, 2020, 46 с.

Укладач

Пуйда В. Я., к. т. н, доцент

Рецензент

Відповідальний за випуск:

Мельник А. О., професор, завідувач кафедри

Дослідження інтегрованого на кристалі мікроконтролера аналого-цифрового перетворювача

МЕТА РОБОТИ: дослідити режими функціонування інтегрованого на кристалі мікроконтролера аналого-цифрового перетворювача (АЦП) з використанням Visual Studio 2019 Community; ознайомитися з підключенням до мікропроцесорної системи (МПС) та функціонуванням символьного LCD індикатора та інтерфейсу UART.

ПОРЯДОК ВИКОНАННЯ РОБОТИ

1. Ознайомитися з ядром, структурою, вузлами АЦП, паралельних портів та UART мікроконтролера **STM32F407VG** (*STM32F407xx.pdf*).
2. Ознайомитися з лабораторним стендом на основі *на відлагоджувального модуля STM32F4Discovery* (*STM32F4DISCOVERY User manual.pdf*).
3. Перевірити свою теоретичну підготовку по контрольних питаннях для самоперевірки; при необхідності скористатися методичними матеріалами та літературними джерелами.
4. Запустити **Visual Studio 2019 Community**.
5. Ознайомитись з елементами екрану відлагоджувача, використанням команд головного меню та підменю, режимами відображення завантаженої програми, операціями модифікації вмісту комірок пам'яті та регістрів.
6. Ознайомитися з прикладом програми.
7. Виконати основні етапи процесу налагодження програми:
 - створити новий проект або завантажити тестовий проект
 - налагодити програму у різних режимах відлагоджувача
 - запустити програму **ModbusFS 2.2**
8. Виконати індивідуальне завдання визначене керівником лабораторної роботи.
9. Захистити лабораторну роботу.

ЗАВДАННЯ

1. Створити проект в **Visual Studio 2019 Community**.
2. В покроковому режимі продемонструвати роботу драйвера АЦП.
3. В покроковому режимі продемонструвати роботу драйвера символьного LCD.
4. В покроковому режимі продемонструвати роботу драйвера UART.
5. Зняти характеристику АЦП, використовуючи COM-порт та програму **ModbusFS 2.2** (термінал COM-порта).

ПИТАННЯ ДЛЯ САМОПЕРЕВІРКИ

1. Внутрішня структура мікроконтролера STM32F407VG.
2. Програмна модель ядра Cortex-F4.
3. Програмна модель мікроконтролера STM32F407VG.
4. АЦП мікроконтролера STM32F407VG.
5. Паралельні порти мікроконтролера STM32F407VG.
6. Порт UART мікроконтролера STM32F407VG.
7. Структура відлагоджувального модуля STM32F4Discovery
8. Основні опції меню відлагоджувача.

ОСНОВНІ ЕТАПИ ВИКОНАННЯ РОБОТИ

1. Запустити **Visual Studio 2019 Community**.
2. Ознайомитися за допомогою інтерактивної довідки, яка входить у склад програмного пакету, з можливостями середовища та його функціями, з елементами графічного інтерфейсу та з режимами відлагодження.
3. Виконати основні етапи процесу налагодження програми в системі mVision:
 - **створити** новий проект **Embedded** або **завантажити** тестовий; порядок створення нового проекту див. скріншоти нижче;
 - налагодити програму у різних режимах.
4. Запустити програму **ModbusFS2.2** (термінал COM-порта).

ОСНОВНІ ЕТАПИ створення проекту Embedded та налагодження програми у Visual Studio 2019 Community

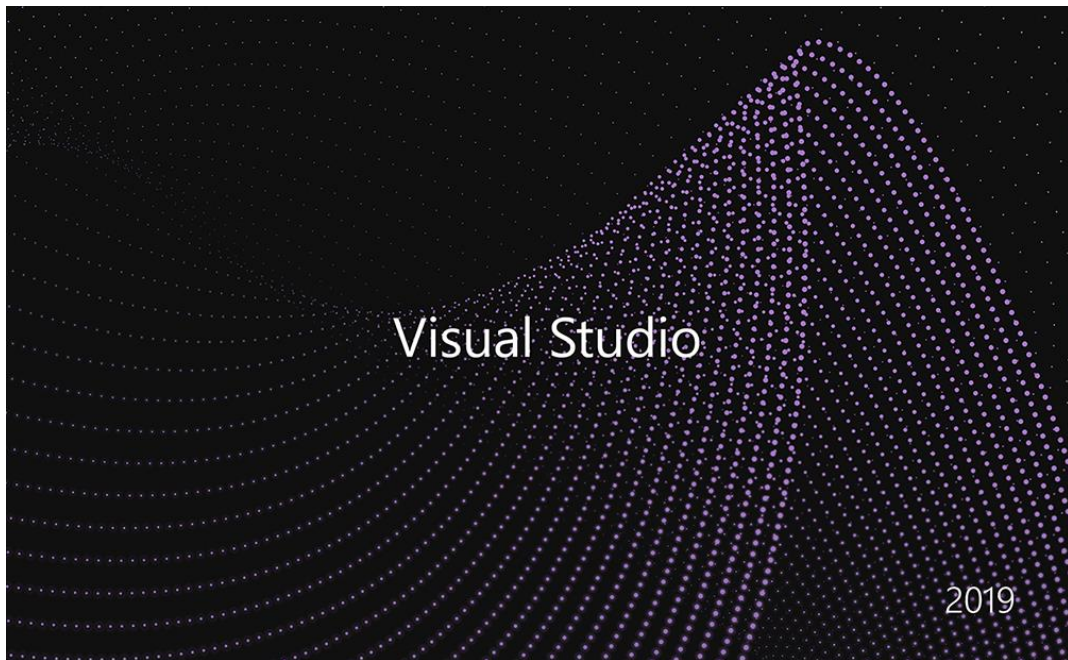


Рис.1. Стартове вікно **Visual Studio 2019 Community**

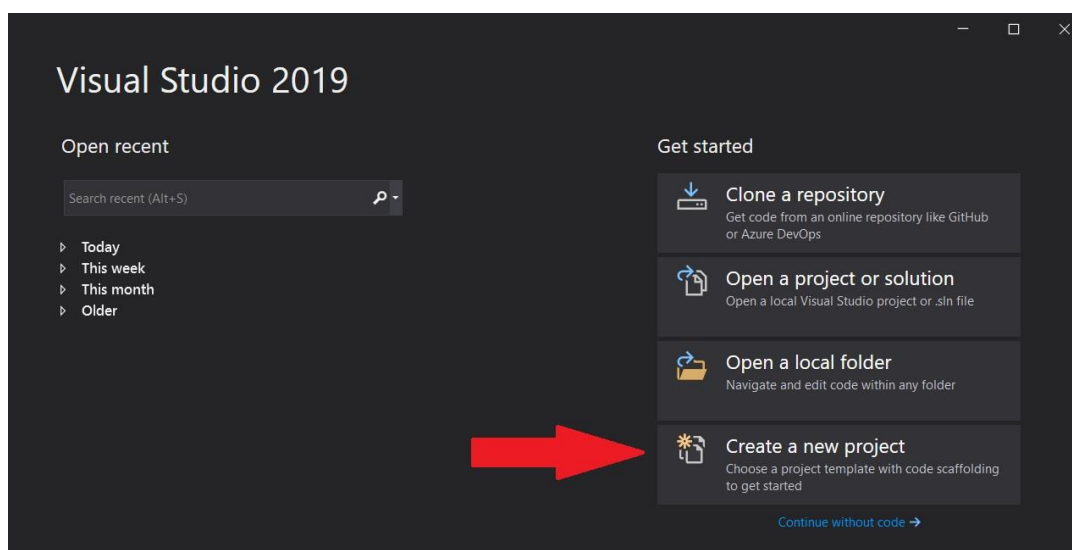


Рис.2. Створення нового проекту

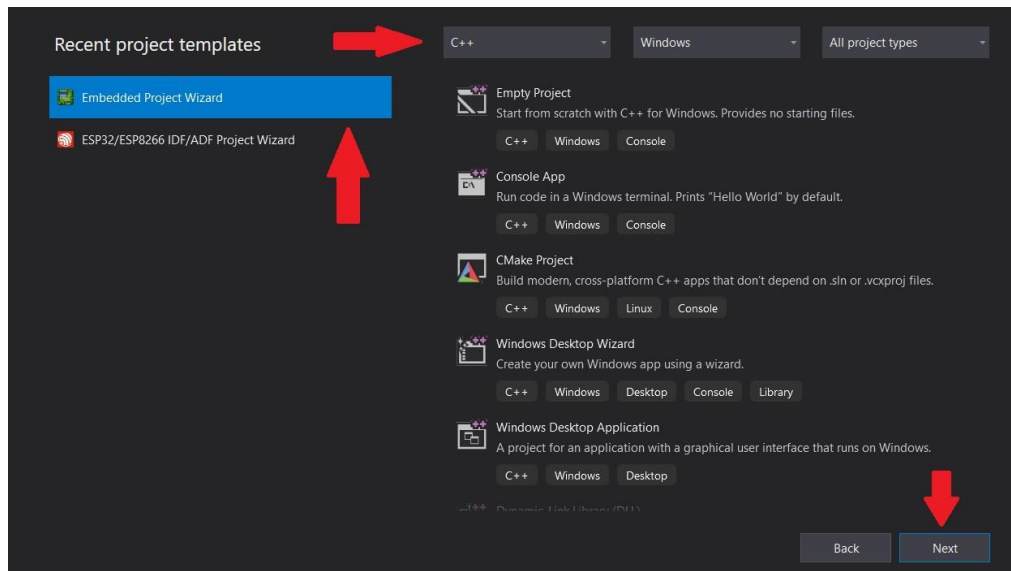
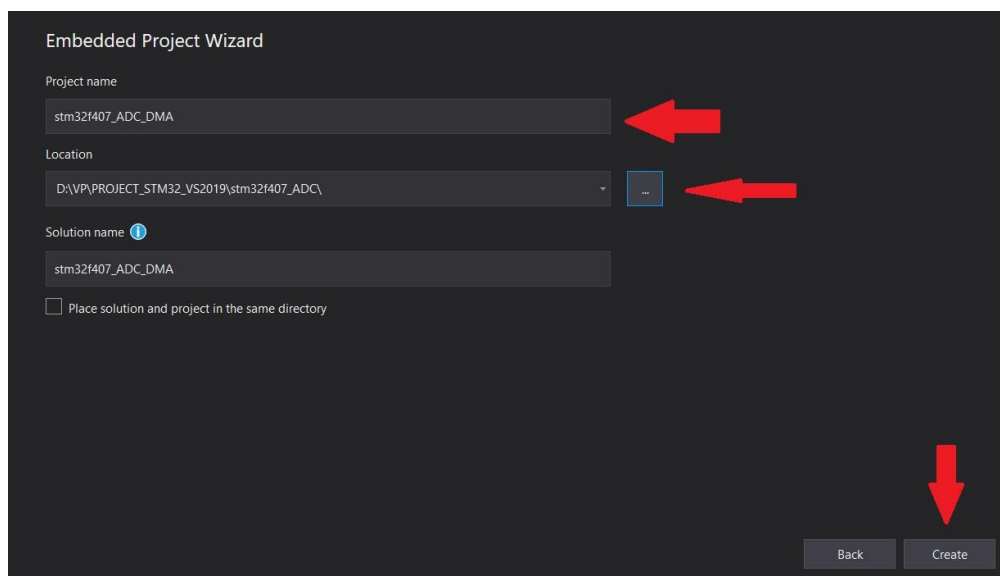
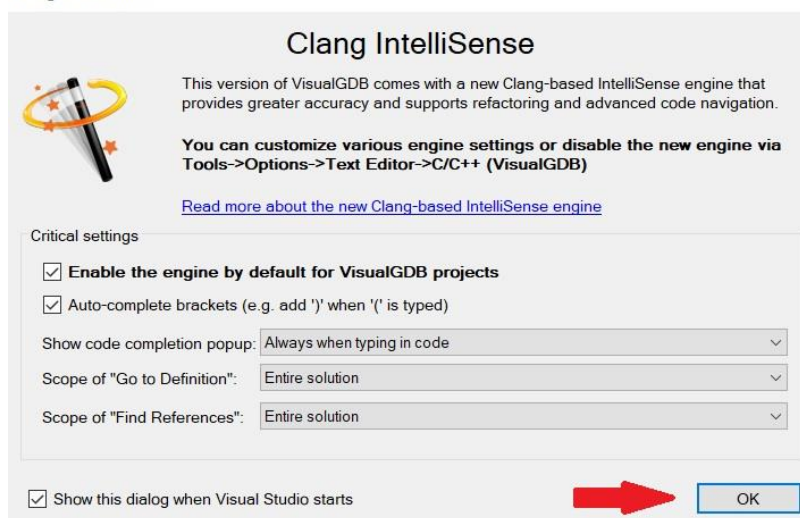


Рис.3. Вибір типу проекту **Embedded** та мови програмування



Clang IntelliSense



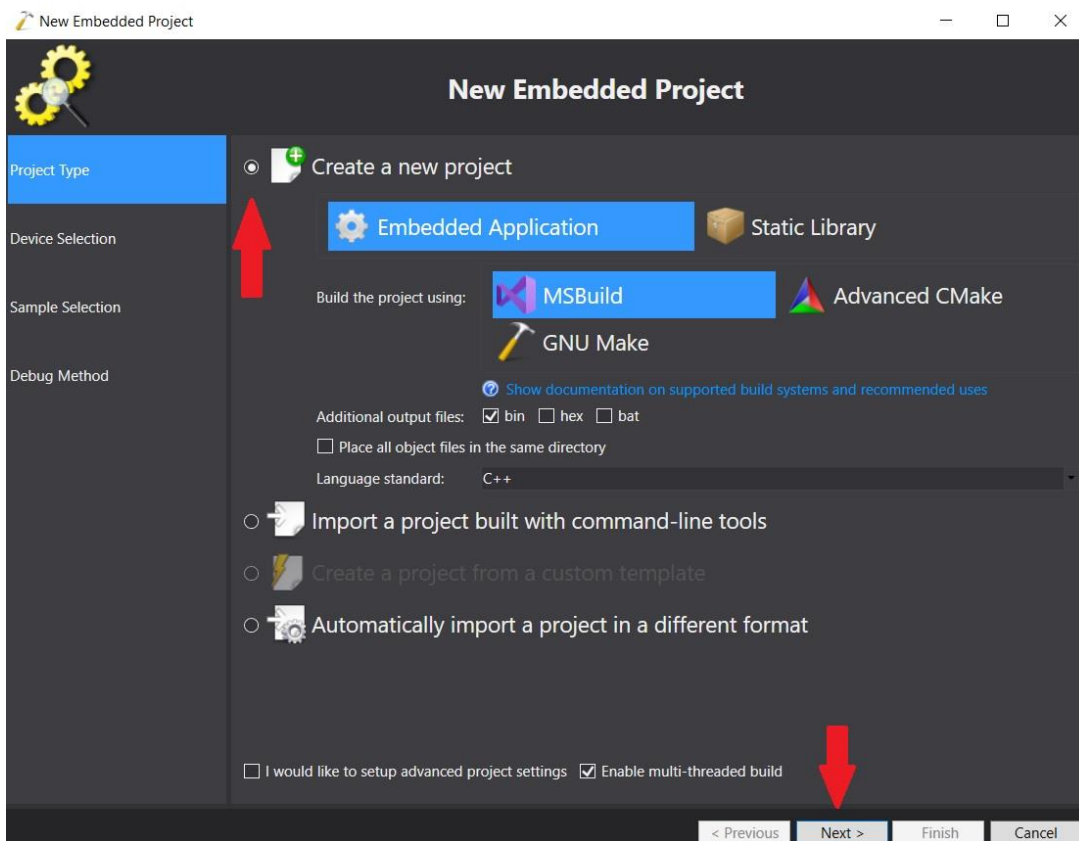


Рис.4. Вибір розміщення та назви проекту

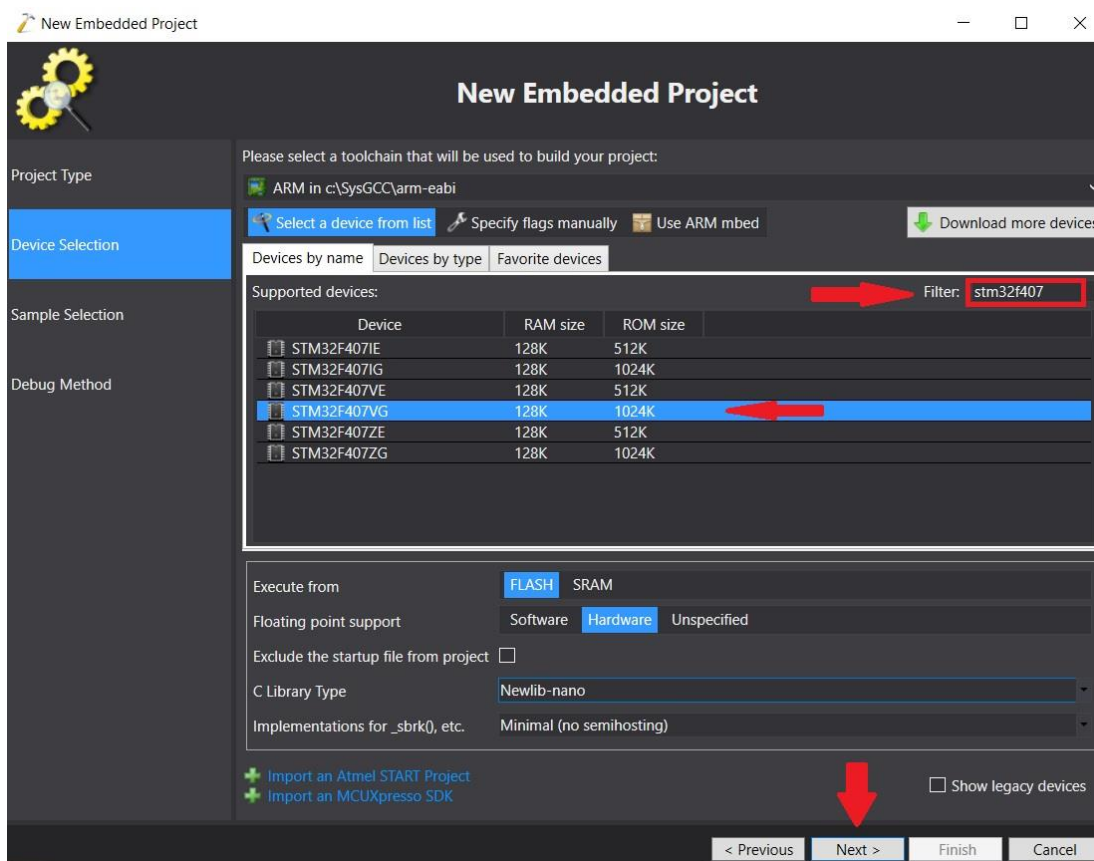


Рис.5. Вибір типу мікроконтролера (STM32F407VG)

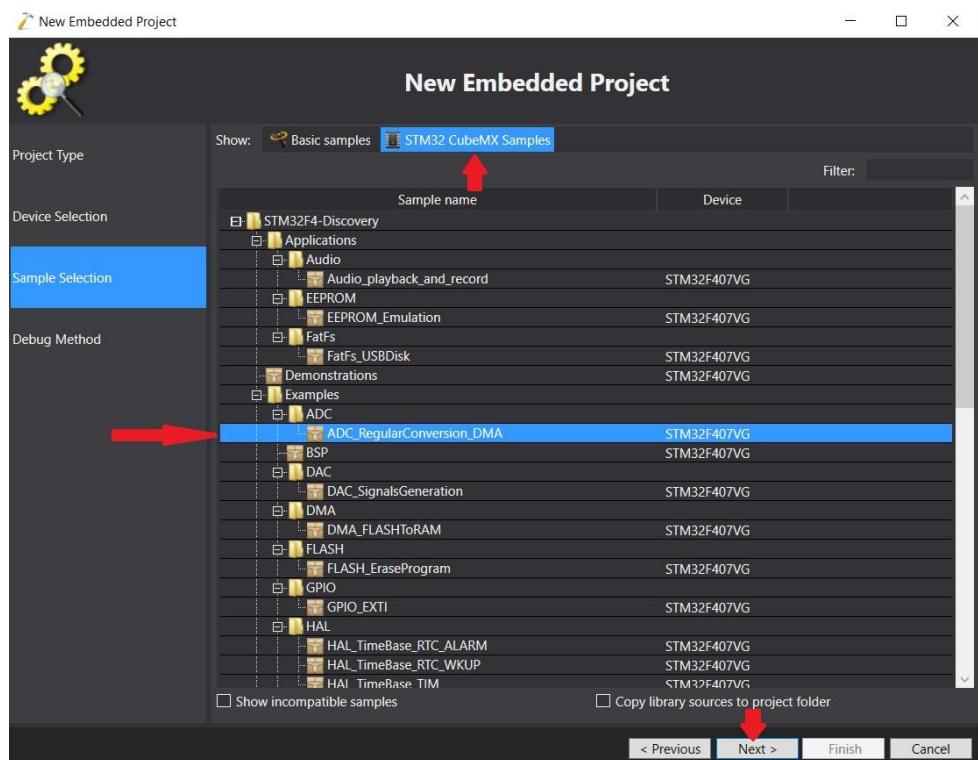


Рис.6. Вибір проекту з переліку прикладів

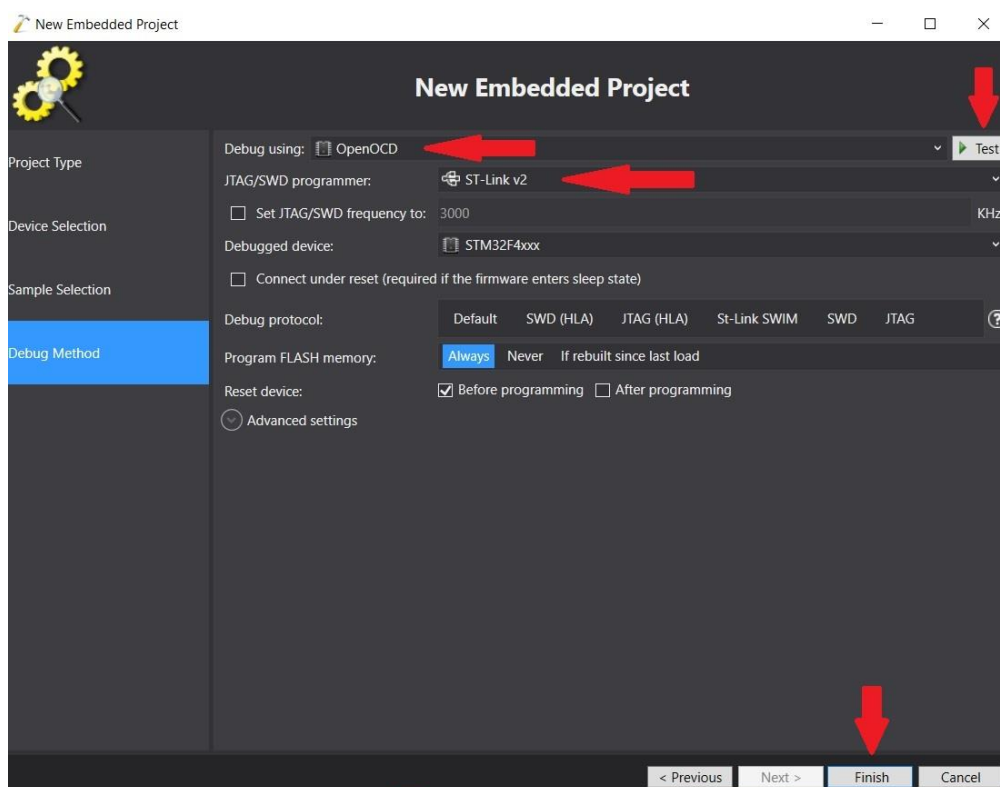


Рис.7. Вибір режимів апаратного відлагоджувача

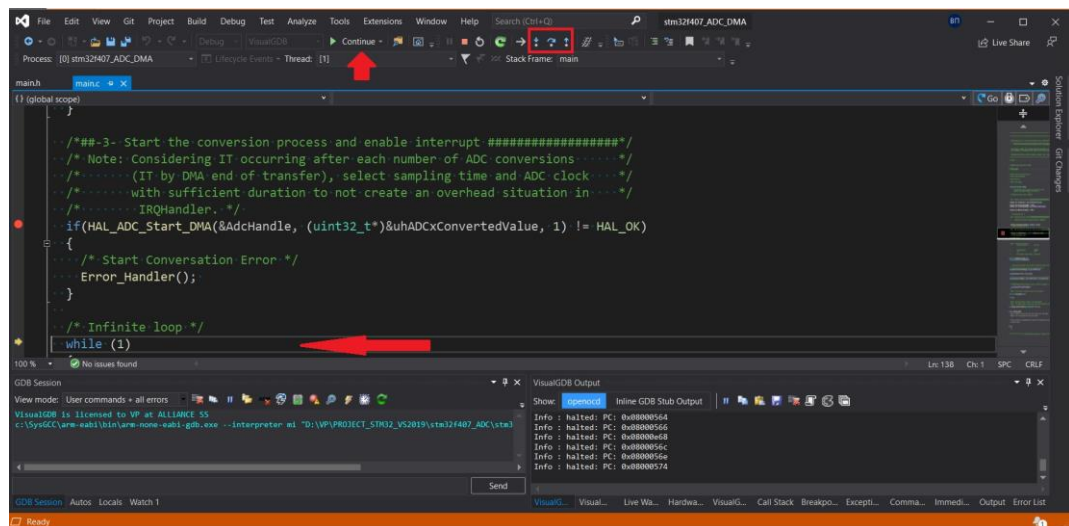
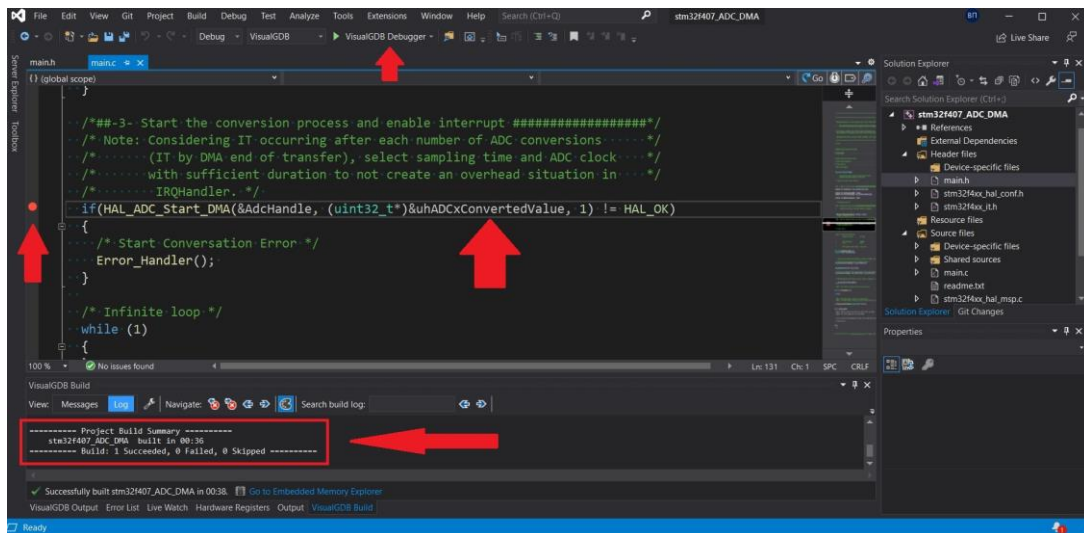
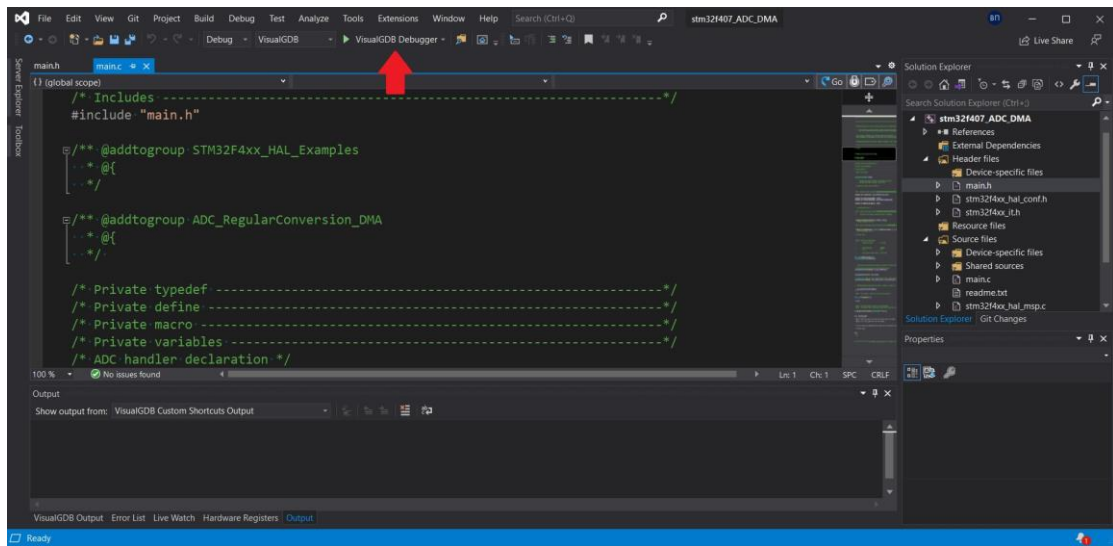


Рис.8. Запуск режиму компілювання та відлагодження програми, встановлення точки переривання

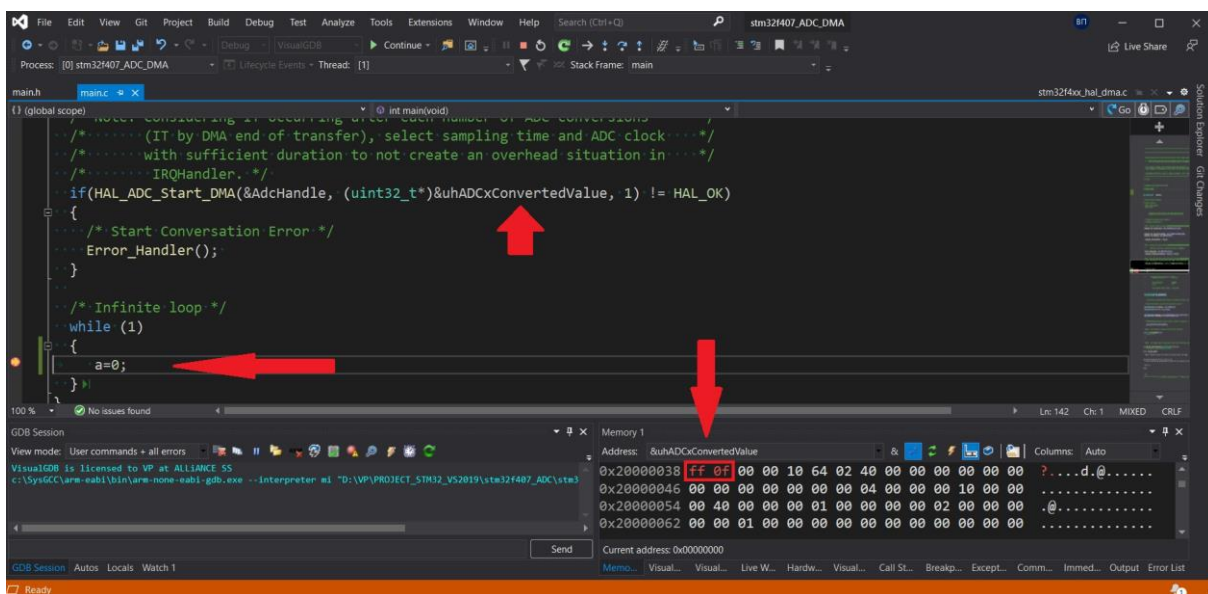
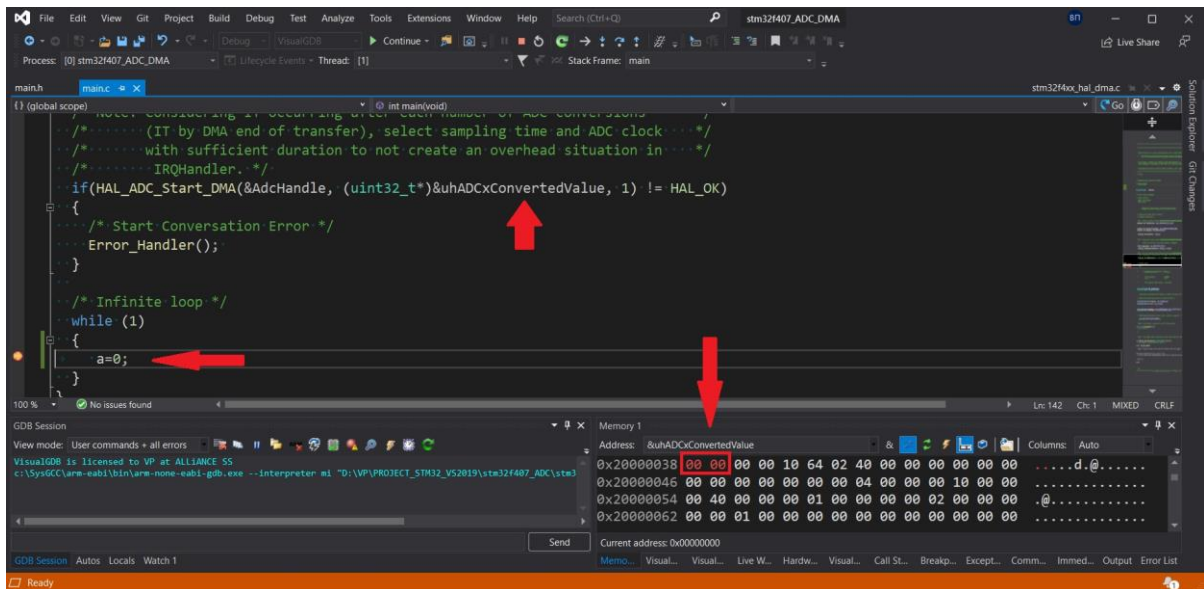


Рис.9. Читання змінної, що зберігає відлік АЦП



Рис.10. Керування та індикація стенду

Прийом значень АЦП в ПК здійснюється програмою «Modbus FS».

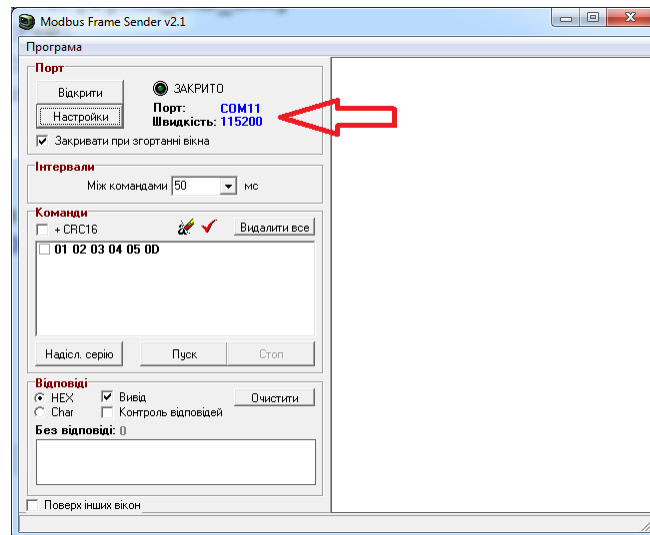


Рис.11. Стартове вікно програми «Modbus FS»

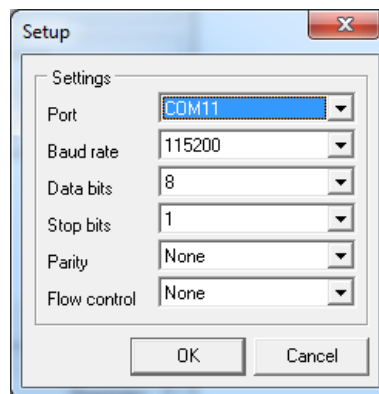


Рис.12. Налаштування програми «Modbus FS»

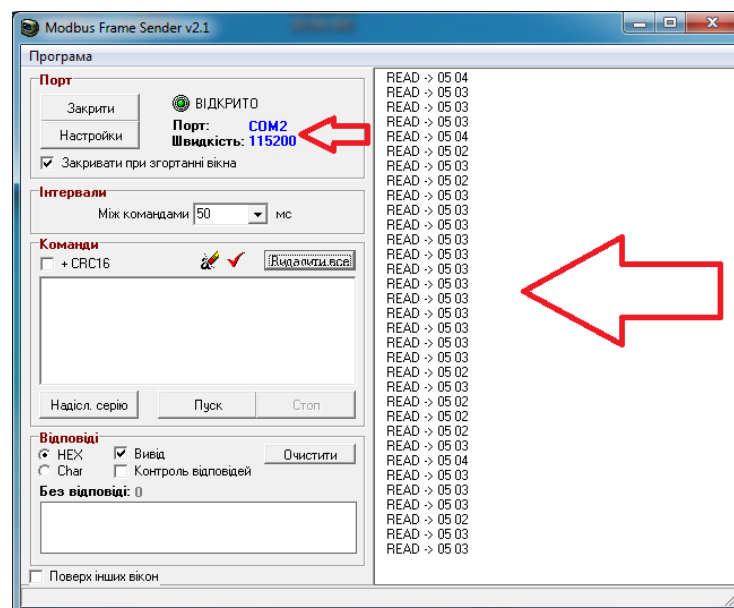


Рис.13. Вивід показів АЦП через UART.

МЕТОДИЧНІ МАТЕРІАЛИ

Лабораторний стенд реалізовано на базі відлагоджувального модуля **STM32F4DISCOVERY**.

Модуль STM32F4DISCOVERY.

Модуль призначений для відлагодження апаратних вузлів мікропроцесорної системи на базі мікроконтролерів STM32F407/417. На платі модуля розміщено мікроконтролер STM32F407VGT6 в корпусі LQFP100 з вузлом синхронізації на базі кварцевого резонатора 8MHz, вузол USB, аудіо-ЦАП, 4 світлодіоди користувача, кнопка RESET, кнопка користувача та контактні групи підключені до виводів мікроконтролера. Крім цього на платі розміщено адаптер ST-LINK/V2 який використовується для відлагодження в реальному часі та програмування пам'яті Flash мікроконтролера. Адаптер може бути відключений від мікроконтролера на платі та використовуватися для відлагодження та програмування плати користувача через інтерфейс SWD.



Рис. 14. Зовнішній вигляд модуля STM32F4DISCOVERY

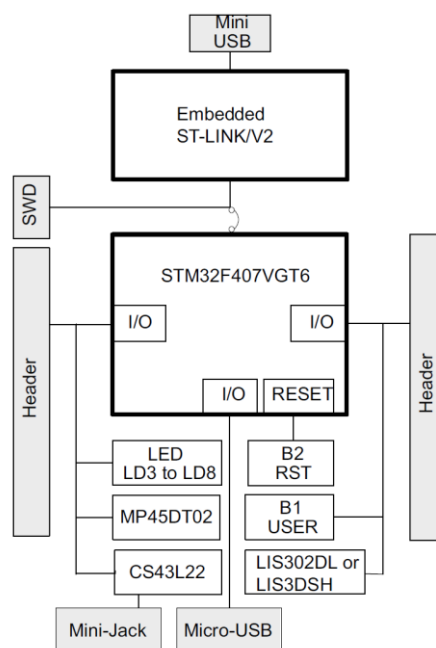


Рис. 15. Структура модуля STM32F4DISCOVERY

На платі модуля розміщено контактні групи **Header**, на які виведено відповідні лінії мікроконтролера STM32F407VGT6, див. таблицю **Table 5. MCU pin description versus board function** у файлі *STM32F4DISCOVERY User manual.pdf*:

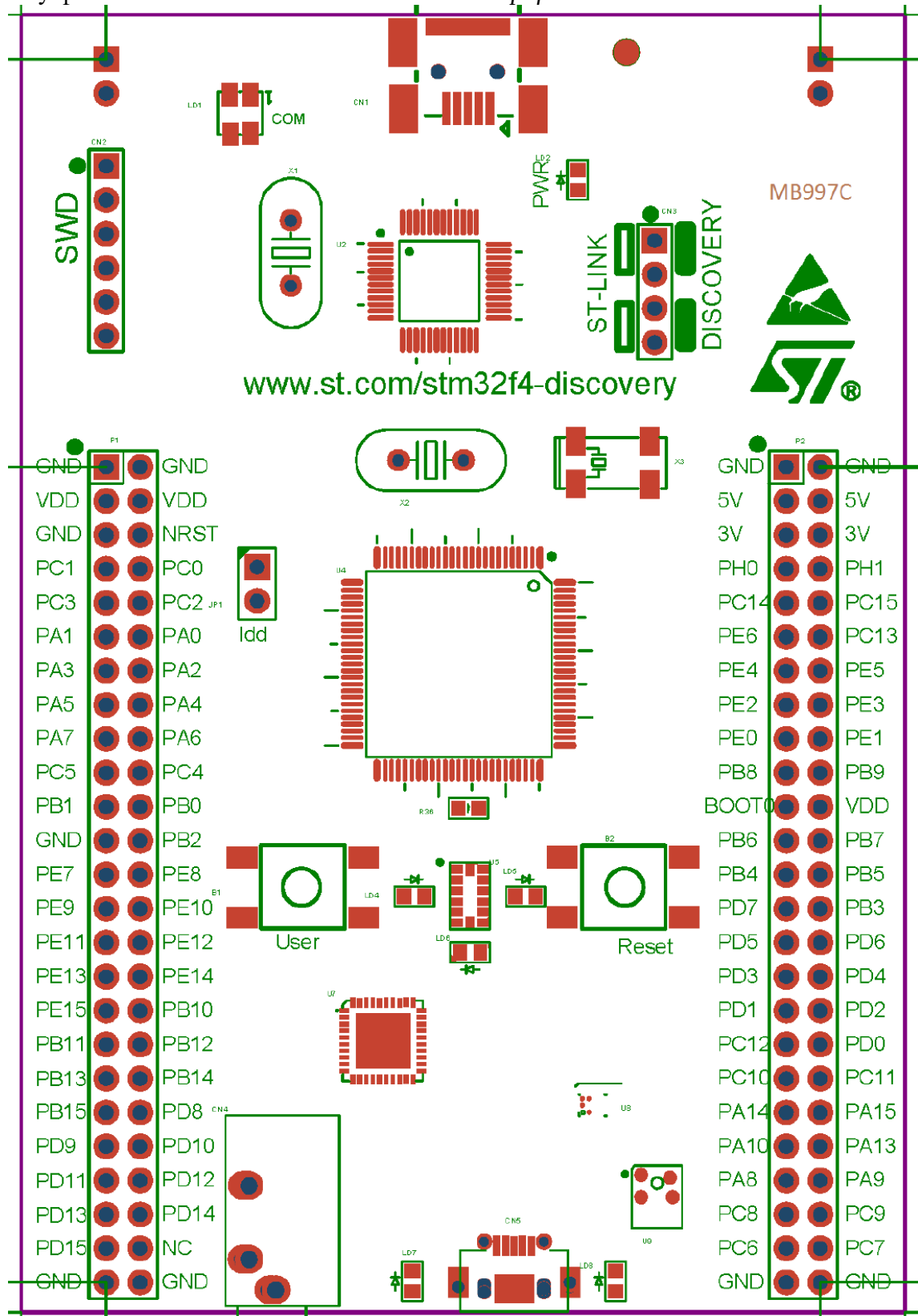


Рис. 16. Контакти **Header** модуля STM32F4DISCOVERY

На платі модуля розміщено відлагоджувальний вузол **ST-LINK/V2**, який забезпечує програмування програмної Flash-пам'яті мікроконтролера та відлагодження в реальному часі записаної програми. Крім цього на платі розміщено **2 кнопки, 4 кольорових світлодіоди LED, Audio DAC, USB, Accelerometer, ST MEMS microphone**, див. *STM32F4DISCOVERY*

Ядро Cortex-M4F

Розроблено для проектування вбудованих мікрокомп'ютерних систем середньої продуктивності з низькою споживною потужністю. Архітектурно це 32-розрядне RISC-ядро типу ARM нового покоління з повним набором інструкцій ARM, реалізацією інструкцій DSP, вбудованим процесором плаваючої коми (FPU) та максимальною робочою частотою до 200 MHz. Ядро має вбудований модуль захисту пам'яті який підвищує безпеку роботи виконання програм. Для оптимального програмування FPU використовується спеціальна мета-мова. Все це дозволяє ефективно обробляти сигнали та реалізовувати складні алгоритми керування.

На базі ядра Cortex-M4F різні фірми освоїли випуск широкої номенклатури мікроконтролерів з різними об'ємами інтегрованої на кристалі пам'яті та різноманітною номенклатурою інтегрованих периферійних пристроїв. Одними з таких мікроконтролерів є мікроконтролери серії STM32F4xx фірми STMicroelectronics які стали досить популярними серед розробників.

Вказані мікроконтролери характеризуються наявністю інтегрованої швидкодіючої пам'яті Flash до 1 Mb та наявністю кеш-пам'яті що забезпечує звертання до пам'яті на максимальній частоті процесора без циклів очікування. Система обробки подій в реальному часі не вимагає втручання процесора що забезпечує обробку без затримок. Номенклатура периферійних пристроїв різних мікроконтролерів поряд з класичними інтерфейсними пристроями включає такі комунікаційні інтерфейси як Ethernet MAC з підтримкою стандарту IEEE1588, CAN, USB, інтерфейси для підключення відеокамери та графічних дисплеїв, 12 та 16-розрядні багатоканальні швидкодіючі аналого-цифрові перетворювачі (АЦП), спеціалізовані цифро-аналогові перетворювачі (ЦАП). Для забезпечення високої продуктивності при обміні даними на кристалах інтегровані канали прямого доступу до пам'яті. Деякі мікроконтролери мають інтегровані на кристалі такі специфічні вузли як криптографічний процесор, модуль обчислення контрольних сум CRC, генератор випадкових чисел.

Важливою особливістю вказаних мікроконтролерів є можливість функціонування в режимі пониженого споживання при батарейному живленні. Також при пропаданні основного живлення забезпечується зберігання важливих даних в SRAM з батарейним живленням та функціонування таймера реального часу. Мікроконтролери випускаються в різних корпусах з різною кількістю виводів від 64 до 176, що дозволяє оптимізувати апаратні засоби при різних застосуваннях. Напруга живлення мікроконтролерів від 1.8V до 3.6V, температура в робочому режимі від - 40 до +105 градусів С.

ВНУТРІШНЯ СТРУКТУРА STM32407.

Основні характеристики

- **ядро 32 розряди;**
- **максимальна частота ядра 168 MHz;**
- **вбудований процесор плаваючої коми FPU;**
- **інструкції ARM та DSP;**
- **вбудована Flash пам'ять програм до 1Mb та SRAM даних до 192 Kb з можливістю зовнішнього розширення;**
- **SRAM 4Kb з зовн. резервним живленням;**
- **широка номенклатура інтегрованої периферії;**
- **VDD = 3.3 V ($1.8\text{ V} \leq \text{VDD} \leq 3.6\text{ V}$)**

Вузли STM32F40x

- **ядро ARM® Cortex™-M4F 168 MHz;**
- **система синхронізації PLL;**
- **інтегрований процесор FPU;**
- **інтегрована Flash пам'ять програм до 1Mb;**
- **інтегрована SRAM даних до 192 Kb;**

- інтегрована резервна SRAM даних 4Kb (Vbat);
- адаптивний акселератор пам'яті (ART Accelerator);
- блок захисту пам'яті (MPU);
- модуль обчислення контрольних сумм (CRC unit);
- генератор випадкових чисел (RNG);
- мульти-АнВ матриця шин 32 р;
- контролер прямого доступу до пам'яті (DMA);
- контролер статичної пам'яті (FSMC) з можливістю конфігурації керування PKI;
- контролер вкладених векторних переривань (NVIC);
- контролер зовнішніх переривань (EXTI);
- широка номенклатура інтерфейсних вузлів для паралельного та послідовного обміну (GPIO, (SDIO)-SD/SDIO/MMC, USART, SPI, I²C, CAN, USB, Ethernet, I2S);
- інтерфейс цифрової камери (DCMI);
- інтегровані 12р АЦП та ЦАП;
- універсальні таймери, RTC, WDT, SysTick ;
- вбудовані макро-реєстри трасування (Embedded Trace Macrocell, ETM);
- JTAG та SW інтерфейси для трасування та програмування інтегрованої Flash;
- інтегрований стабілізатор напруги живлення;
- вузол керування живленням ;

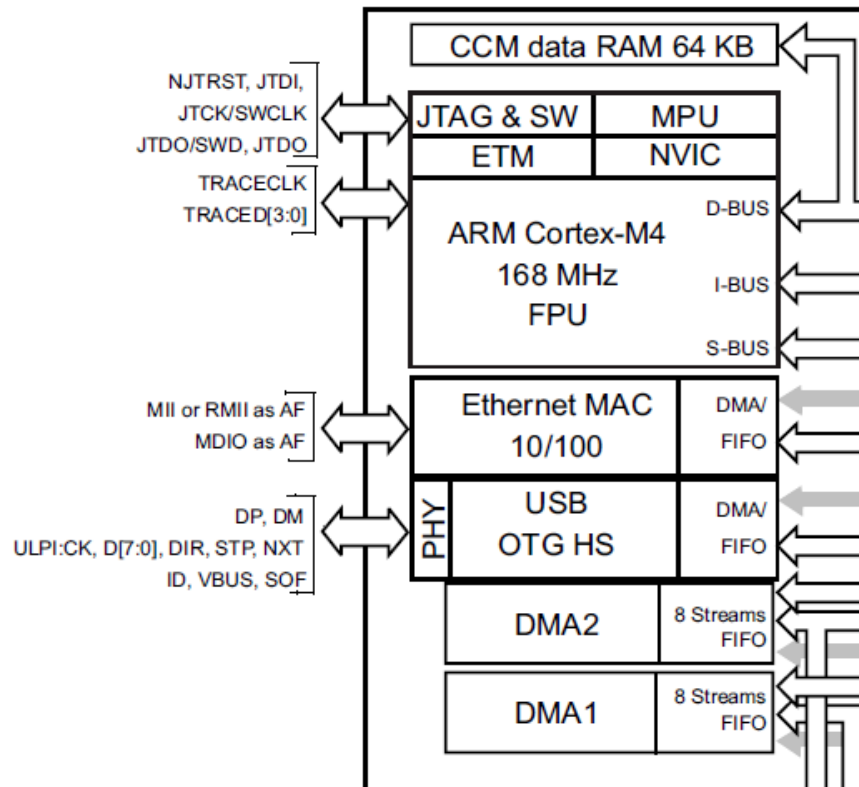


Рис.17а. Внутрішня структура STM32F407

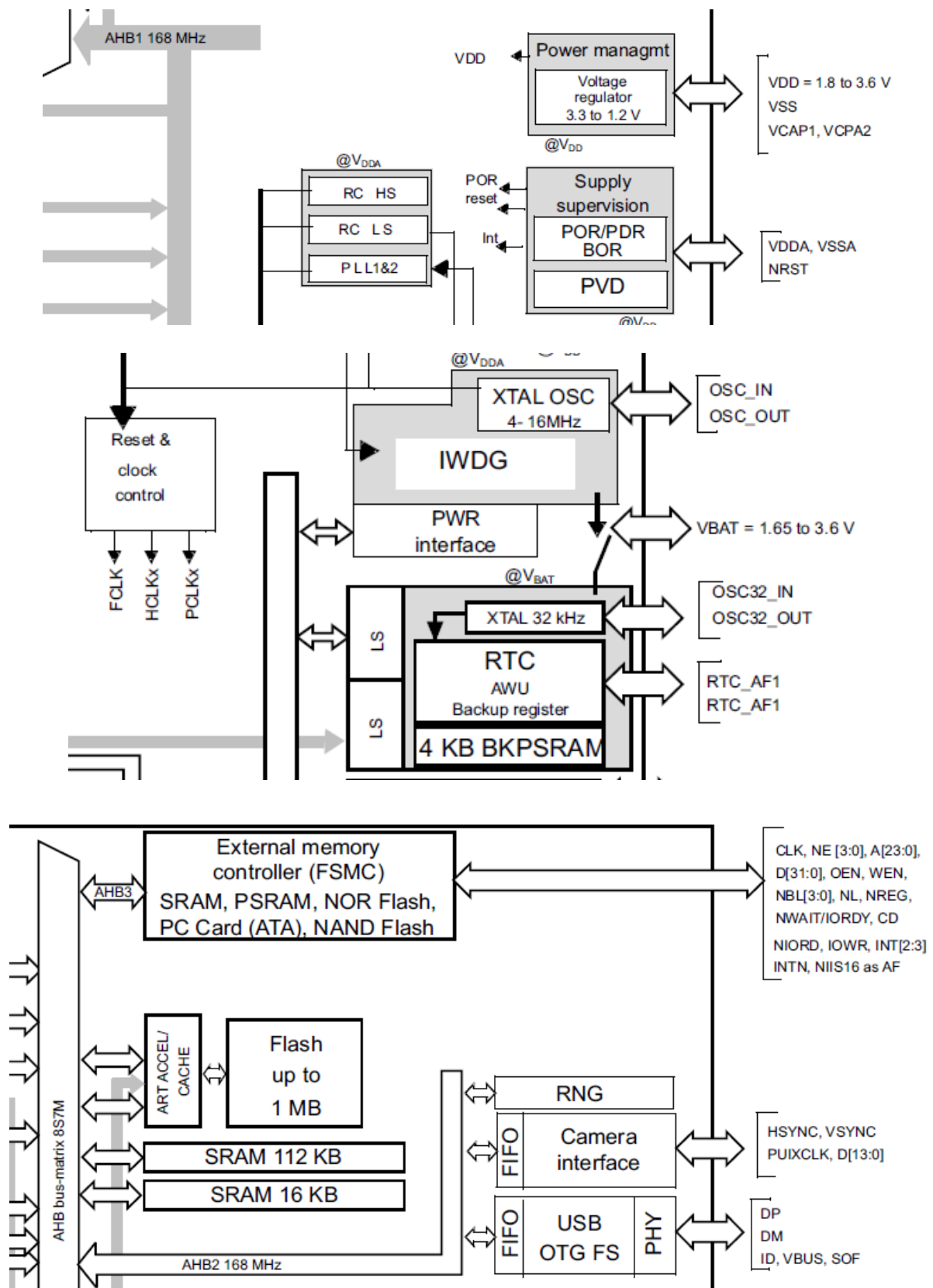


Рис.176. Внутрішня структура STM32F407

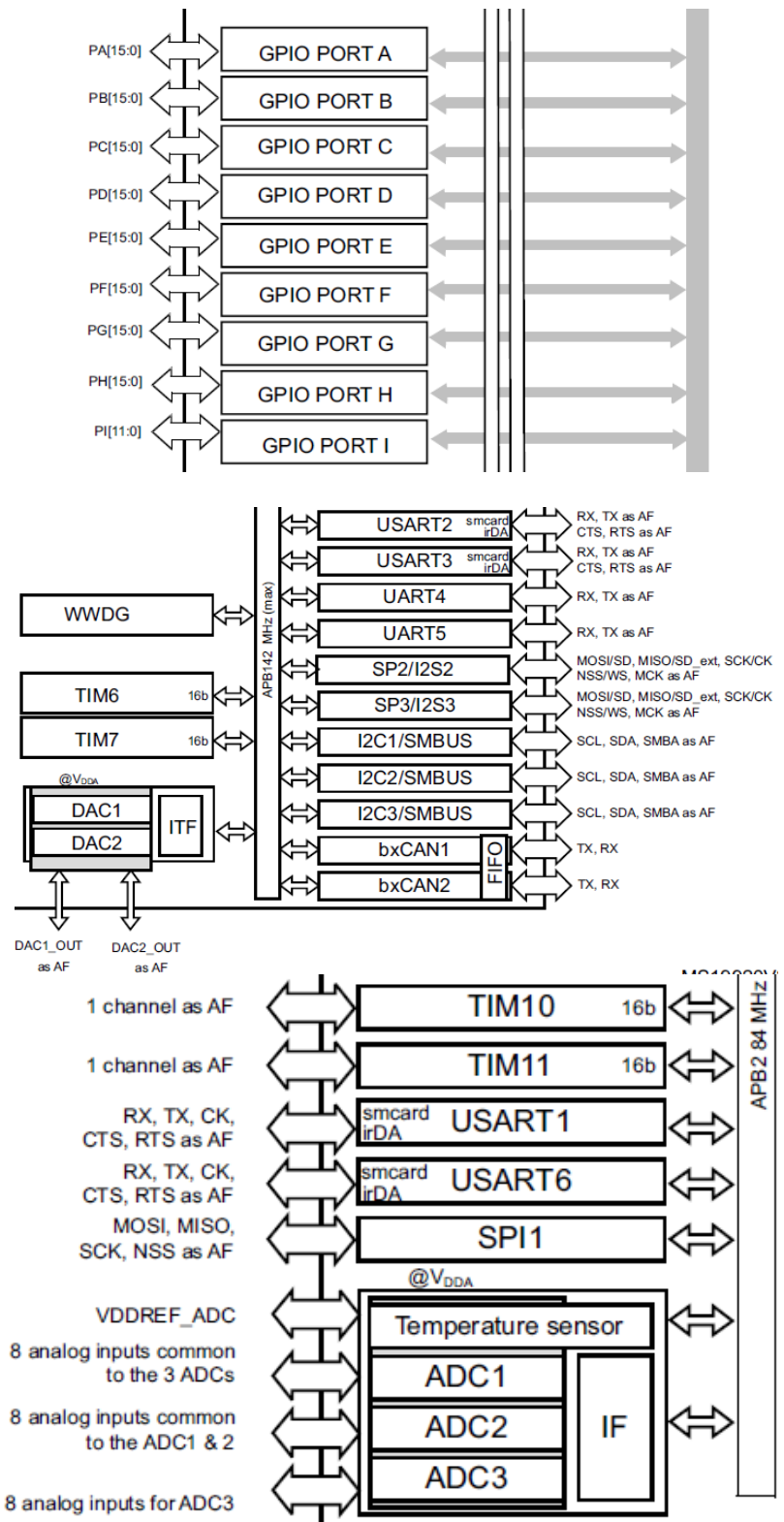


Рис.17в. Внутрішня структура STM32F407

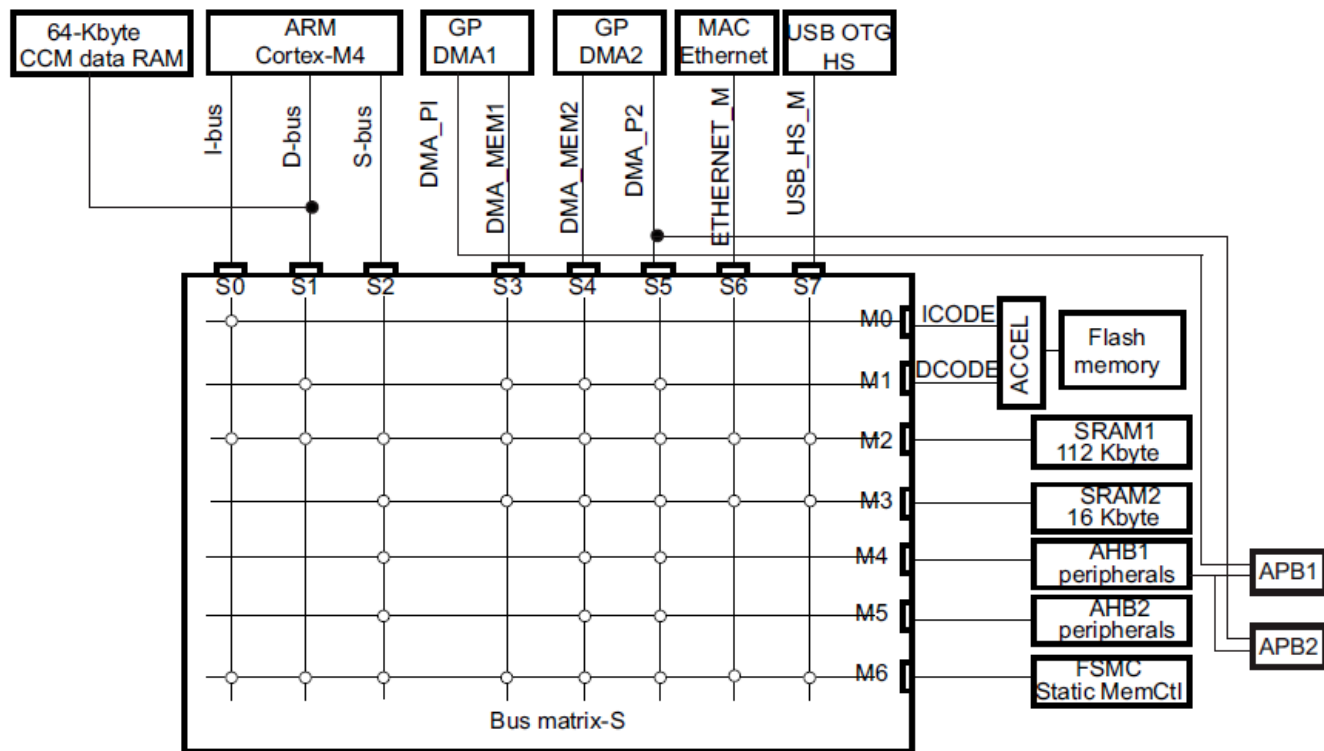


Рис.18. Мульти-АВБ матрица шин 32 р

КПДП (DMA) memory-to-memory, peripheral-to-memory and memory-to-peripheral

- SPI and I2S
- I2C
- USART
- General-purpose, basic and advanced-control timers TIMx
- DAC
- SDIO
- Camera interface (DCMI)
- ADC.

Живлення

- VDD = 1.8 to 3.6 V: external power supply for I/Os and the internal regulator (when enabled), provided externally through VDD pins.
- VSSA, VDDA = 1.8 to 3.6 V: external analog power supplies for ADC, DAC, Reset blocks, RCs and PLL. VDDA and VSSA must be connected to VDD and VSS, respectively.
- VBAT = 1.65 to 3.6 V: power supply for RTC, external clock 32 kHz oscillator and backup registers (through power switch) when VDD is not present.

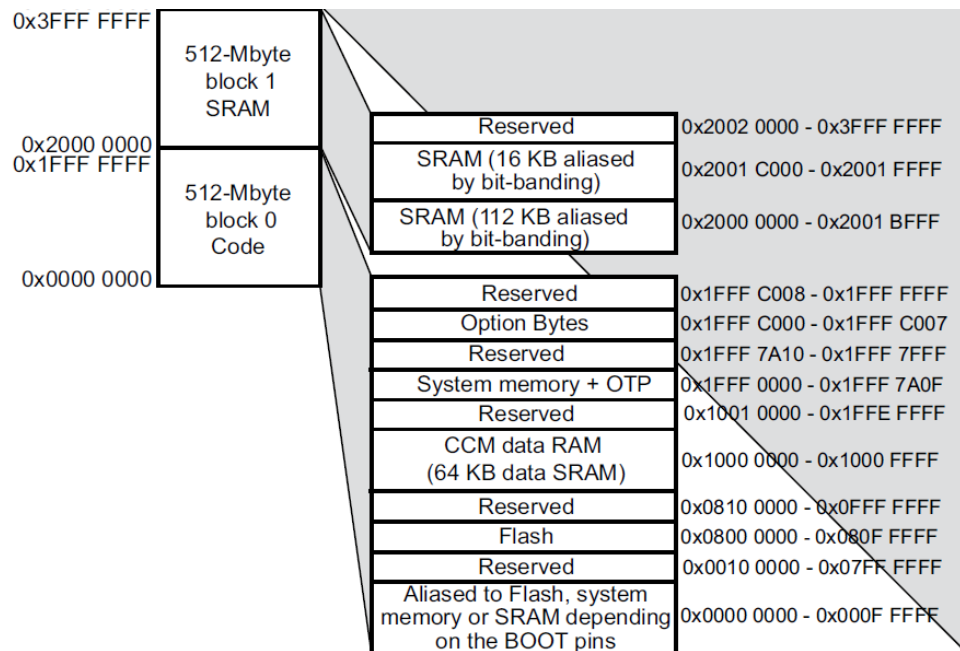


Рис.19а. Структура пам'яті STM32F40x

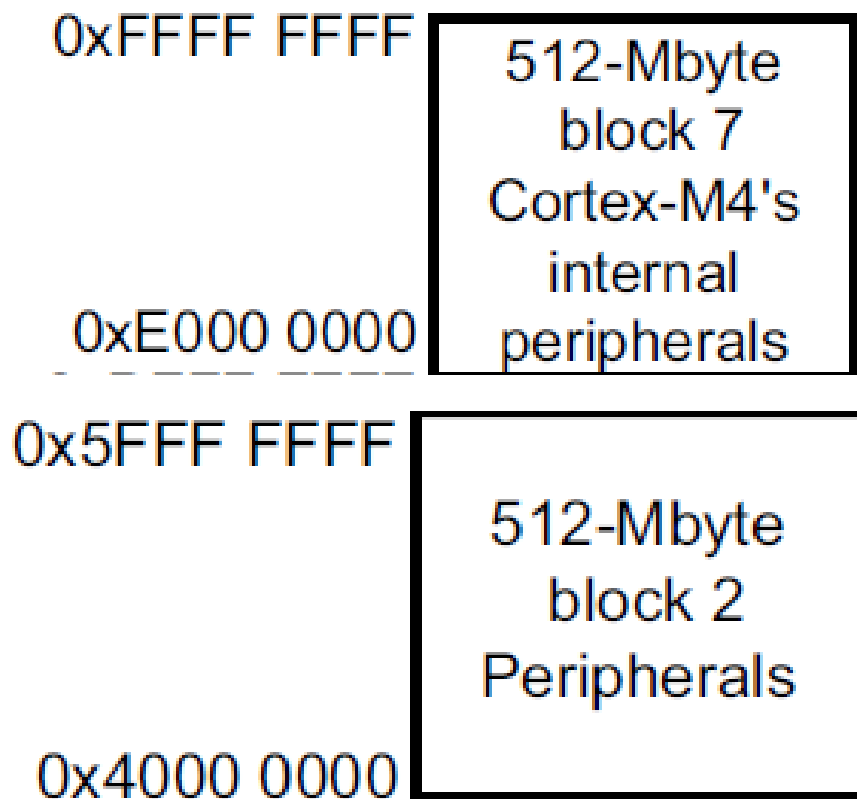


Рис.19b. Структура пам'яті STM32F40x

Bus	Boundary address	Peripheral
	0xE00F FFFF - 0xFFFF FFFF	Reserved
Cortex-M4	0xE000 0000 - 0xE00F FFFF	Cortex-M4 internal peripherals
	0xA000 1000 - 0xDFFF FFFF	Reserved
AHB3	0xA000 0000 - 0xA000 0FFF	FSMC control register
	0x9000 0000 - 0x9FFF FFFF	FSMC bank 4
	0x8000 0000 - 0x8FFF FFFF	FSMC bank 3
	0x7000 0000 - 0x7FFF FFFF	FSMC bank 2
	0x6000 0000 - 0x6FFF FFFF	FSMC bank 1
	0x5006 0C00- 0x5FFF FFFF	Reserved
AHB2	0x5006 0800 - 0x5006 0BFF	RNG
	0x5005 0400 - 0x5006 07FF	Reserved
	0x5005 0000 - 0x5005 03FF	DCMI
	0x5004 0000- 0x5004 FFFF	Reserved
	0x5000 0000 - 0x5003 FFFF	USB OTG FS
	0x4008 0000- 0x4FFF FFFF	Reserved

Рис.20а. Регістри STM32F40x

Bus	Boundary address	Peripheral
AHB1	0x4004 0000 - 0x4007 FFFF	USB OTG HS
	0x4002 9400 - 0x4003 FFFF	Reserved
	0x4002 9000 - 0x4002 93FF	ETHERNET MAC
	0x4002 8C00 - 0x4002 8FFF	
	0x4002 8800 - 0x4002 8BFF	
	0x4002 8400 - 0x4002 87FF	
	0x4002 8000 - 0x4002 83FF	
	0x4002 6800 - 0x4002 7FFF	Reserved
	0x4002 6400 - 0x4002 67FF	DMA2
	0x4002 6000 - 0x4002 63FF	DMA1
	0x4002 5000 - 0x4002 5FFF	Reserved
	0x4002 4000 - 0x4002 4FFF	BKPSRAM
	0x4002 3C00 - 0x4002 3FFF	Flash interface register
	0x4002 3800 - 0x4002 3BFF	RCC
	0x4002 3400 - 0x4002 37FF	Reserved
	0x4002 3000 - 0x4002 33FF	CRC
	0x4002 2400 - 0x4002 2FFF	Reserved
	0x4002 2000 - 0x4002 23FF	GPIOI
	0x4002 1C00 - 0x4002 1FFF	GPIOH
	0x4002 1800 - 0x4002 1BFF	GPIOG
	0x4002 1400 - 0x4002 17FF	GPIOF
	0x4002 1000 - 0x4002 13FF	GPIOE
	0x4002 0C00 - 0x4002 0FFF	GIPOD
	0x4002 0800 - 0x4002 0BFF	GPIOC
	0x4002 0400 - 0x4002 07FF	GPIOB
	0x4002 0000 - 0x4002 03FF	GPIOA
	0x4001 5800- 0x4001 FFFF	Reserved

Рис.20b. Регістри STM32F40x

АЦП мікроконтролера STM32F407

STM32F407 має в своєму складі три 12-бітних аналого-цифрових перетворювачі (АЦП), кожний з яких може отримувати вхідну напругу через спеціально призначені (мультиплексовані) входи і перетворювати їх в числовий код. Вхідна напруга порівнюється з опорною напругою **Reference Voltage (VREF)**. Опорна напруга може братися або з аналогового живлення **VDDA** або з інтегрованого чи зовнішнього джерела опорної напруги **VREF**.

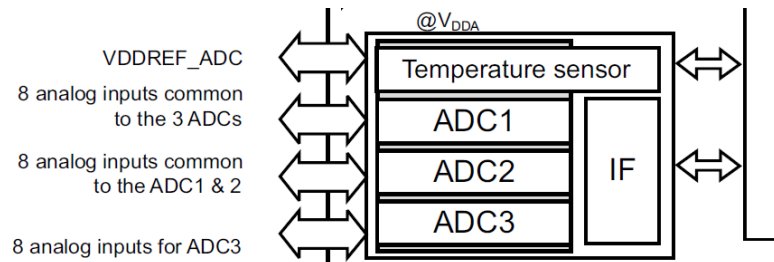


Рис.21. АЦП мікроконтролера STM32F407

Крім зовнішніх входів доступні також кілька внутрішніх каналів: канал внутрішнього температурного сенсора (V_{sense}), канал для внутрішньої опорної напруги (V_{refint}), канал для моніторингу зовнішньої напруги живлення батареї (V_{bat}). Перетворення різних каналів може виконуватися в одноканальному (**single**), багатоканальному (**scan**), безперервному (**continuous**) або переривчастому (**discontinuous**) режимах. Результат роботи АЦП записується в 16-розрядний регістр даних з лівим або правим вирівнюванням. Крім того, АЦП також може працювати в якості аналогового сторожового таймера, який дозволяє програмі визначати момент, коли вхідна напруга виходить за рамки заданого користувачем діапазону і, якщо це відбувається, генерується переривання.

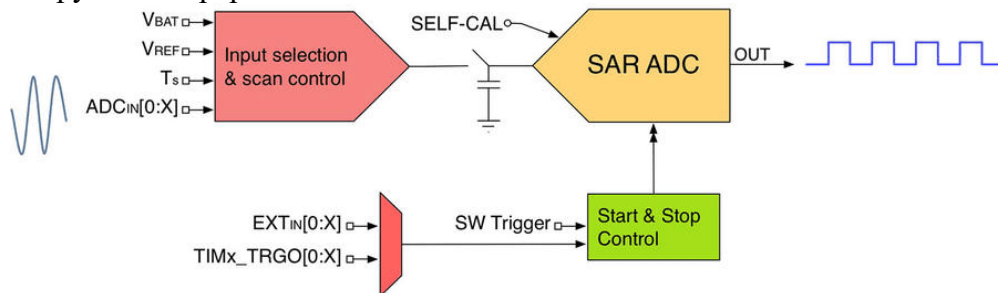


Рис. 22. Структура АЦП

Блок вибору каналу і контролю сканування (**Input Selection & Scan Control**) виконує вибір джерела вхідної напруги для АЦП. Залежно від режиму перетворення (**single**, **scan** або **continuous**), цей блок автоматично перемикає вхідні канали, так що кожен з них може бути зчитаний з певним періодом. Вихід даного блоку підключений до АЦП (**SAR ADC**). На Рис. 2 зображено блок контролю запуску і зупинки (**Start&Stop&Control**). Його роль полягає в контролі процесу перетворення АЦП, він може перемикаватися як програмно, так і по подіях від деяких вхідних джерел. Крім того, внутрішньо він підключений до лінії TRGO декількох таймерів, тому може бути реалізовано перетворення за таймером в режимі DMA.

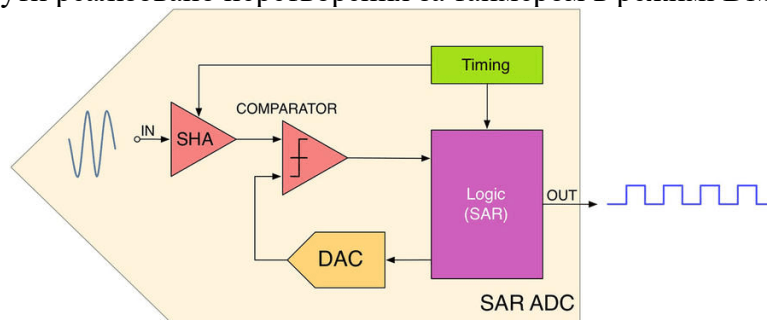


Рис. 23. Внутрішня структура АЦП

На Рис. 3 показано основні вузли: вузол вибірки і зберігання (**Sample-and-Hold SHA**), компаратор (**Comparator**), цифро-аналоговий перетворювач (ЦАП), (**DAC**) та вузол логіки (**Logic SAR**), який обчислює цифрове значення коду по алгоритму послідовного наближення. Алгоритм послідовного наближення обчислює напругу вхідного сигналу шляхом її порівняння з величиною, що генерується від внутрішнього ЦАП (DAC): якщо вхідний сигнал більше, ніж ця внутрішня опорна напруга, то відбувається подальше збільшення цієї опорної напруги поки вхідний сигнал не стане менше за неї. Остаточний результат співвідноситься з числовим представленням в межах від нуля до максимального значення 12 бітного цілого беззнакового числа, тобто $2^{12} - 1 = 4095$. Якщо прийняти що $V_{REF} = 3300\text{мВ}$ то 3300мВ представляється числом 4095. Це означає що 1 одиниця АЦП рівна $3300/4095 = 0.8\text{мВ}$. Нехай вхідний сигнал дорівнює 2.5В , тоді він буде перетворений в цифровий код **3102** в десятковому представленні:

$$x = \frac{4095}{3300\text{мВ}} \times 2500\text{мВ} = 3102$$

Режими перетворення

АЦП має кілька режимів роботи, які можуть використовуватися в різних сценаріях роботи програми

Один канал, одноразовий режим перетворення.

Це найпростіший режим роботи АЦП. У цьому режимі, АЦП здійснює одне перетворення (одна вибірка) одного каналу, як показано на Рис 4, і завершує свою роботу, коли перетворення завершено.

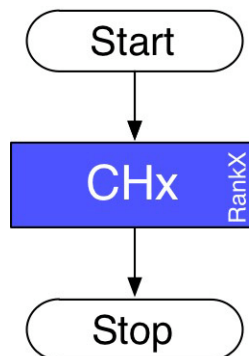


Рис. 24. Один канал, одне перетворення

Декілька каналів, одноразовий режим перетворення (багатоканальний режим)

Цей режим використовується для отримання по одній вибірці з кожного каналу в незалежному режимі. Можна задати будь-яку послідовність каналів АЦП з різним часом перетворення і різної черговості, але ви не можете зупинити АЦП під час процесу перетворення для того, щоб змінити час перетворення наступного каналу. Цей режим додатково навантажує процесорне ядро, складний програмно і його краще здійснювати в режимі канал прямого доступу (DMA).

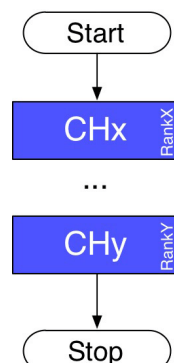


Рис. 25. Багатоканальний режим

Один канал, режим безперервного перетворення.

Цей режим дозволяє перетворювати один канал безперервно і нескінченно довго в режимі перетворення регулярних каналів. Безперервний режим роботи дозволяє АЦП працювати фонові. АЦП перетворює значення одного каналу без будь-якого втручання з боку процесора. В циклічному режимі може бути використаний DMA, що зменшує навантаження на процесорне ядро.

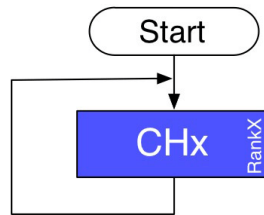


Рис. 26. Режим циклічного перетворення одного каналу

Режим циклічного перетворення декількох каналів.

Цей режим також називають багатоканальним безперервним режимом і він може використовуватися для перетворення декількох каналів в незалежному режимі. Використовуючи індекси каналів **Rank**, доступна настройка будь-якої послідовності каналів з різним часом вибірок і черговістю перетворення. Цей режим схожий на простий багатоканальний режим з тією лише різницею, що перетворення не зупиняється після перетворення останнього каналу в послідовності регулярних каналів, а починається по новій з самого першого каналу і триває цей цикл нескінченно довго. У цьому режимі також доступний DMA.

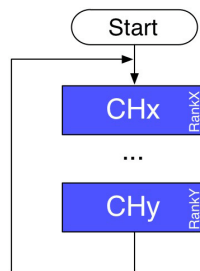


Рис. 27. Режим циклічного перетворення декількох каналів

Режим перетворення інжектованих каналів.

Цей режим призначений для використання в режимі запуску перетворення по зовнішній події або програмно. Група інжектованих каналів має більш високий пріоритет ніж група регулярних каналів. Її перетворення може перервати перетворення поточного каналу в групі регулярних каналів.

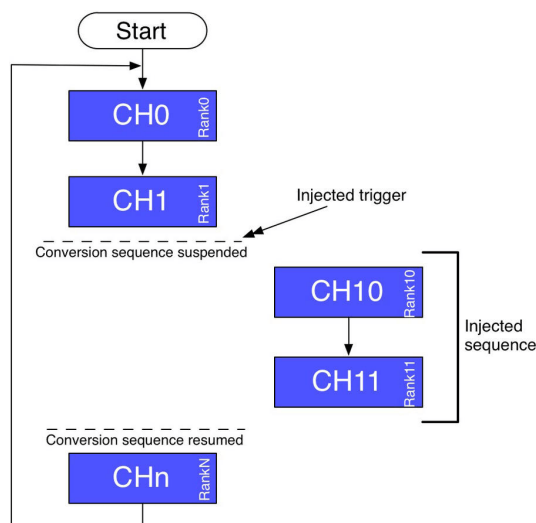


Рис. 28. Режим перетворення інжектованих каналів

Вибір каналу

Вхідні канали фіксовані і прив'язані до конкретних виводів мікроконтролера (IN0, IN1 і т.д.). В той же час вони можуть бути логічно змінені на будь-яку черговість вибірок. Перепризначення каналів реалізовано шляхом призначення кожному каналу індексу в діапазоні від 1 до 16. Цей індекс називається Rank в CubeHAL і відповідає черговості перетворення каналу в послідовності регулярних каналів.

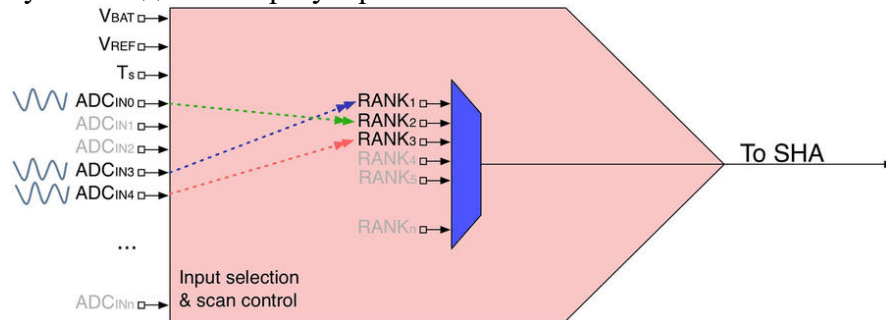


Рис. 29. Переназначення каналів через індекс RANK

Конфігурація канал / індекс представлена структурою `C ADC_ChannelConfTypeDef`, яка визначена таким способом:

```
typedef struct {
uint32_t Channel;          /* Канал, якому назначается индекс Rank */
uint32_t Rank;             /* Индекс черговости канала */
uint32_t SamplingTime;     /* Час одної вибірки для вибраного каналу */
uint32_t Offset;          /* Зарезервовано, может бути встановлено в 0 */
} ADC_ChannelConfTypeDef;
```

- **Channel:** встановлює ідентифікатор каналу; може приймати значення `ADC_CHANNEL_0`, `ADC_CHANNEL_1` ... `ADC_CHANNEL_N` в залежності від доступного числа каналів.
- **Rank:** відповідає індексу черговості для конкретного каналу.
- **SamplingTime:** встановлює час одної вибірки для каналу, визначається числом тактів АЦП та вибирається з списку конкретних значень.

```
/*##-2-Configure ADC regular channel #####*/
/* Note: Considering IT occurring after each number of size of .....*/
/* ..... "uhADCxConvertedValue" ADC conversions (IT by DMA end .....*/
/* ..... of transfer), select sampling time and ADC clock with sufficient .....*/
/* ..... duration to not create an overhead situation in IRQHandler, .....*/
sConfig.Channel = ADCx_CHANNEL;
sConfig.Rank = 1;
sConfig.SamplingTime = ADC_SAMPLETIME_28CYCLES;
sConfig.Offset = 0;

if (HAL_ADC_ConfigChannel(&AdcHandle, &sConfig) != HAL_OK)
{
/* Channel Configuration Error */
Error_Handler();
}
```

Рис. 30. Конфігурація каналів АЦП

```

/* Definition for ADCx Channel Pin */
#define ADCx_CHANNEL_PIN ..... GPIO_PIN_0
#define ADCx_CHANNEL_GPIO_PORT ..... GPIOB

/* Definition for ADCx's Channel */
#define ADCx_CHANNEL ..... ADC_CHANNEL_8

/* Definition for ADCx's DMA */
#define ADCx_DMA_CHANNEL ..... DMA_CHANNEL_0
#define ADCx_DMA_STREAM ..... DMA2_Stream0

/* Definition for ADCx's NVIC */
#define ADCx_DMA_IRQn ..... DMA2_Stream0_IRQn
#define ADCx_DMA_IRQHandler ..... DMA2_Stream0_IRQHandler

```

Рис. 31. Параметри для налаштування АЦП

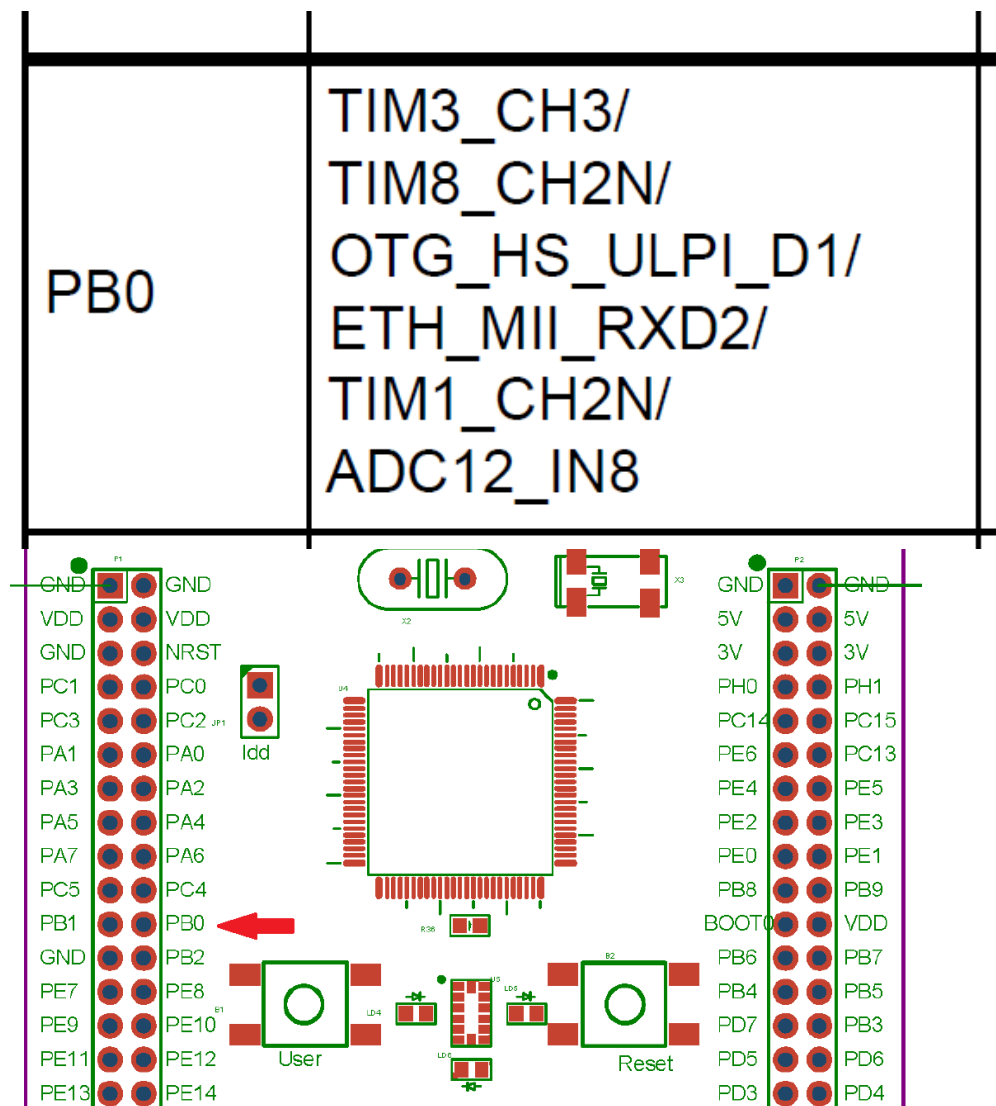


Рис. 32. Контакт PB0 каналу IN8 АЦП

Багаторазове неперервне перетворення в режимі DMA

Для виконання багаторазового перетворення в режимі DMA:

- встановити параметр `hadc.Init.DMAContinuousRequests` в **ENABLE**.
- викликати `HAL_ADC_Start_DMA()` для запуску перетворення.

Якщо замість цього встановити `hadc.Init.DMAContinuousRequests` в **DISABLE**, то необхідно буде кожен раз викликати `HAL_ADC_Stop_DMA ()` в кінці кожної послідовності перетворень, а після знову викликати `HAL_ADC_Start_DMA ()`. В іншому випадку перетворення не буде виконано.

```
·/*##-1-·Configure·the·ADC·peripheral·#####*/  
·AdcHandle.Instance = ADCx;  
·  
·AdcHandle.Init.ClockPrescaler = ADC_CLOCKPRESCALER_PCLK_DIV2;  
·AdcHandle.Init.Resolution = ADC_RESOLUTION_12B;  
·AdcHandle.Init.ScanConvMode = DISABLE;  
·AdcHandle.Init.ContinuousConvMode = ENABLE;  
·AdcHandle.Init.DiscontinuousConvMode = DISABLE;  
·AdcHandle.Init.NbrOfDiscConversion = 0;  
·AdcHandle.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;  
·AdcHandle.Init.ExternalTrigConv = ADC_EXTERNALTRIGCONV_T1_CC1;  
·AdcHandle.Init.DataAlign = ADC_DATAALIGN_RIGHT;  
·AdcHandle.Init.NbrOfConversion = 1;  
·AdcHandle.Init.DMAContinuousRequests = ENABLE;  
·AdcHandle.Init.EOCSelection = DISABLE;  
·.....  
·if(HAL_ADC_Init(&AdcHandle) != HAL_OK)  
·{  
·····/*·Initialization·Error·*/  
·····Error_Handler();  
·}
```

Рис. 33. Конфігурація режимів та параметрів АЦП

```
·/*##-3-·Start·the·conversion·process·and·enable·interrupt·#####*/  
·/*·Note:·Considering·IT·occurring·after·each·number·of·ADC·conversions·.....*/  
·/*·.....(IT·by·DMA·end·of·transfer),·select·sampling·time·and·ADC·clock·.....*/  
·/*·.....with·sufficient·duration·to·not·create·an·overhead·situation·in·.....*/  
·/*·.....IRQHandler·.....*/  
·if(HAL_ADC_Start_DMA(&AdcHandle, (uint32_t*)&uhADCxConvertedValue, 1) != HAL_OK)  
·{  
·····/*·Start·Conversion·Error·*/  
·····Error_Handler();  
·}
```

Рис. 34. Запуск перетворення АЦП в режимі DMA

Вузол відображення на базі алфавітно-цифрового LCD модуля BC1602A

В процесі виконання програми перетворення АЦП результат перетворення виводиться на символний індикатор BC1602A та передається в комп'ютер через порт UART.

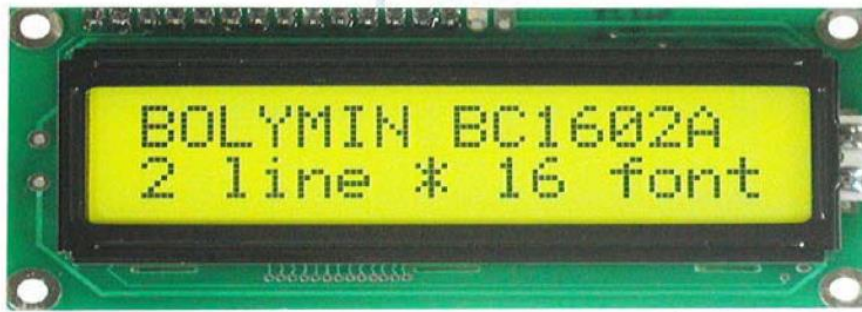


Рис.35. Алфавітно-цифровий LCD модуль BC1602A фірми Bolymin

Pin	Symbol	Function
1	Vss	GND
2	Vdd	+5V(+3V option)
3	Vo	Contrast adjustment
4	RS	H/L register select signal
5	R/W	H/L read/write signal
6	E	H --> L enable signal
7~14	DB0~7	H/L data bus line
15	A/Vee	Power supply for B/L (+)
16	K	Power supply for B/L (GND)

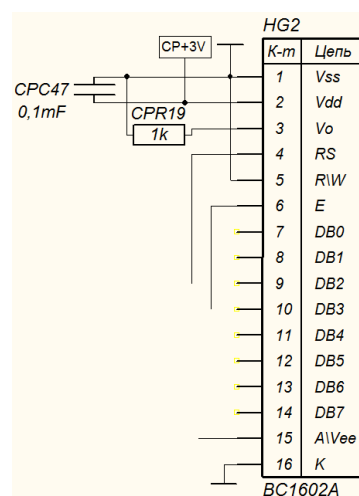


Рис.36. Контакти BC1602A

ТАБЛИЦЯ кодів символів BC1602A

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0				0	1	2	3	4	5	6	7	8	9	A	B	C
1			!	2	3	4	5	6	7	8	9	A	B	C	D	E
2			"	3	4	5	6	7	8	9	A	B	C	D	E	F
3			#	4	5	6	7	8	9	A	B	C	D	E	F	
4			\$	5	6	7	8	9	A	B	C	D	E	F		
5			%	6	7	8	9	A	B	C	D	E	F			
6			&	7	8	9	A	B	C	D	E	F				
7			'	8	9	A	B	C	D	E	F					
8			(	9	A	B	C	D	E	F						
9			)	A	B	C	D	E	F							
A			*	:	J	Z	j	z			Ф	К	„	↓	ё	ѐ
B			+	;	K	[	k	]			Ч	л	”	ћ	џ	ѣ
C			,	<	L	ф	l	ѐ			Ш	М	џ	ћ	ij	xl
D			-	=	M	]	m	is			Ъ	Н	Ї	Н	•	џ
E			.	>	N	^	n	ѐ			Ы	П	џ	џ	џ	џ
F			/	?	O	_	o	ѐ			Э	Т	ѐ	•	о	■

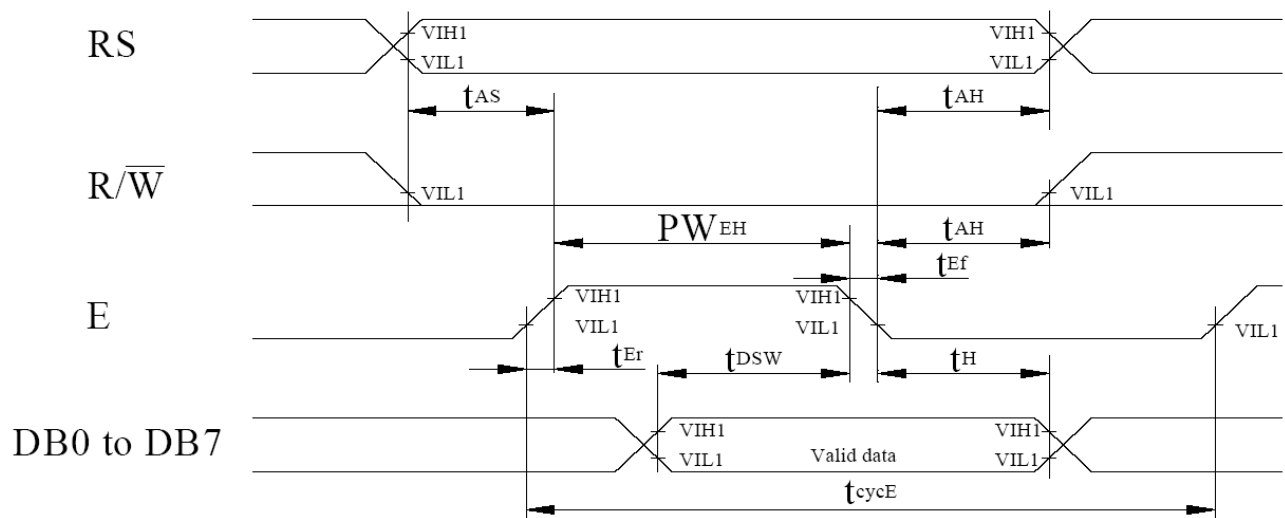


Рис.37. Часова діаграма запису символу в індикатор

BC1602A – процедура ініціалізації

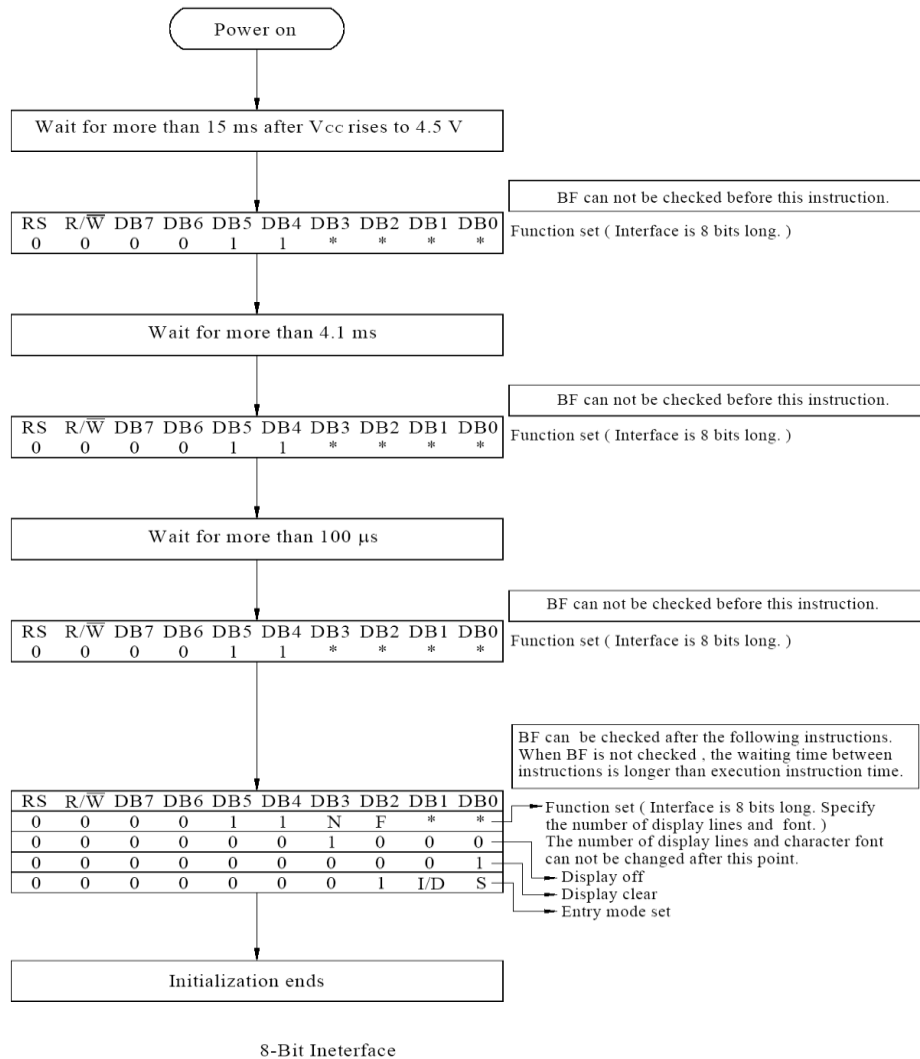
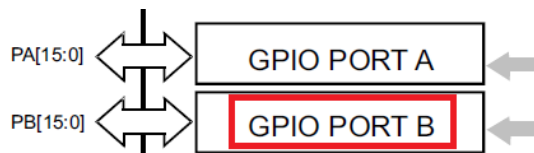


Рис.38. Процедура ініціалізації BC1602A

Керування режимами індикатора BC1602A здійснюється за допомогою інтегрованого на платі індикатора контролера типу HD44780, див. файл *hd44780u.pdf*.

Символьний індикатор BC1602A підключено через лінії паралельного порту GPIO PORT B (PB) в режимі 4-х ліній даних, див. Table 5. MCU pin description versus board function (continued) документу *STM32F4DISCOVERY User manual.pdf*



```
//LCD1602
#define LCM_OUT .....GPIOB->ODR

#define LCM_PIN_RS .....GPIO_PIN_8 .....//PB8
#define LCM_PIN_EN .....GPIO_PIN_1 .....//PB1
#define LCM_PIN_D7 .....GPIO_PIN_7 .....//PB7
#define LCM_PIN_D6 .....GPIO_PIN_6 .....//PB6
#define LCM_PIN_D5 .....GPIO_PIN_5 .....//PB5
#define LCM_PIN_D4 .....GPIO_PIN_4 .....//PB4
```

Рис.38. Підключення BC1602A через лінії порту PB

```

__GPIOB_CLK_ENABLE();

GPIO_InitTypeDef GPIO_InitStructure;

GPIO_InitStructure.Pin = GPIO_PIN_1 | GPIO_PIN_4 | GPIO_PIN_5 |
→ ..... GPIO_PIN_6 | GPIO_PIN_7 | GPIO_PIN_8;
GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_HIGH;
GPIO_InitStructure.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOB, &GPIO_InitStructure);

```

Рис.39. Ініціалізація ліній порту PB

Регістри паралельних портів

GPIO Registers

Configuration Registers

STM32 містить чотири регістри конфігурації для кожного з портів:

- Port mode register – GPIOx_MODER
- Output type register – GPIOx_OTYPER
- Speed register – GPIOx_OSPEEDR
- Pull-up/Pull-down register – GPIOx_PUPDR

Кожен з цих регістрів має довжину 32 біти, хоча і не всі біти використовуються у всіх регістрах.

Port Mode Register – GPIOx_MODER

Цей 32-розрядний регістр має 2 бітні значення, які визначають режим роботи.

Можна встановити такі режими:

Bit Values	Description
00	Input
01	General purpose output
10	Alternate function
11	Analog

Port	Reset Value
A	0xA800 0000
B	0x0000 0280
All others	0x0000 0000

Output Type Register – GPIOx_OTYPER

Bit Value	Description
0	Push/pull output

1 Open drain output

Speed Register – GPIOx_OSPEEDR

Bit Values	Description
00	2 MHz Low speed
01	25 MHz Medium speed
10	50 MHz Fast speed
11	100 MHz High speed on 30pF 80 MHz High speed on 15 pF

Pull-up/Pull-down register – GPIOx_PUPDR

Bit Values	Description
00	No pull-up or pull-down resistor
01	Pull-up resistor
10	Pull-down resistor
11	Reserved

Input data register – GPIOx_IDR

Bit Value	Description
0	High logic value
1	Low logic value

Output data register – GPIOx_ODR

Bit Value	Description
0	High logic value
1	Low logic value

Вузол UART

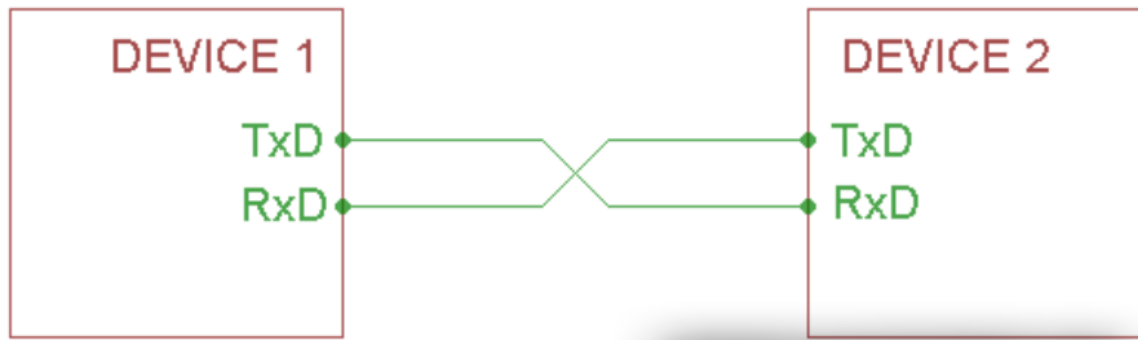
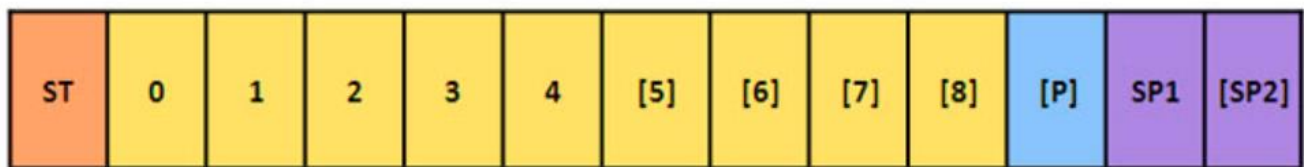


Рис.40. Обмін даними між пристроями через порти UART



- ST – стартовий біт
- 0-8 – біти даних
- P – біт парності
- SP1, SP2 – стопові біти

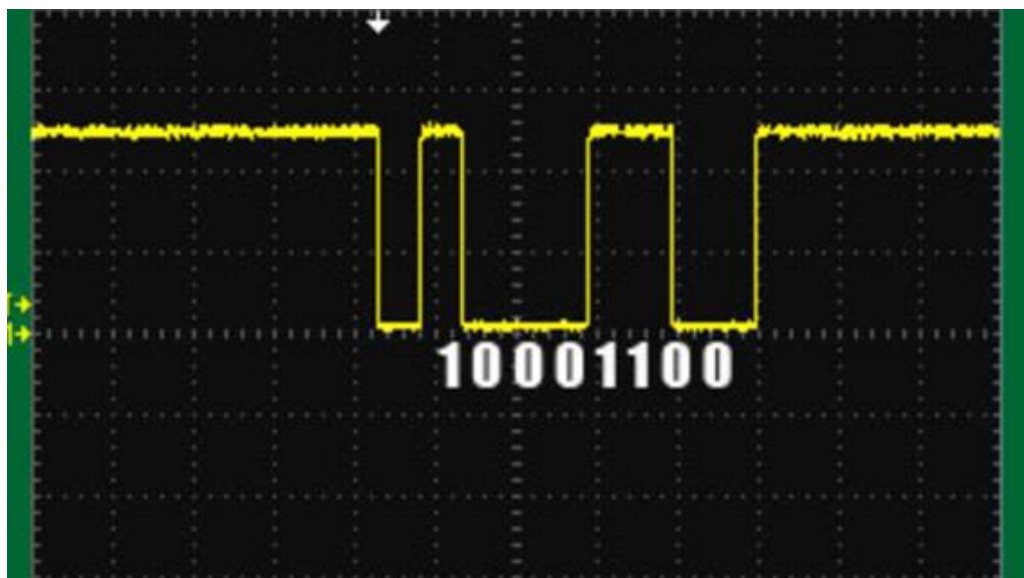


Рис.41. Передача байту через порт UART

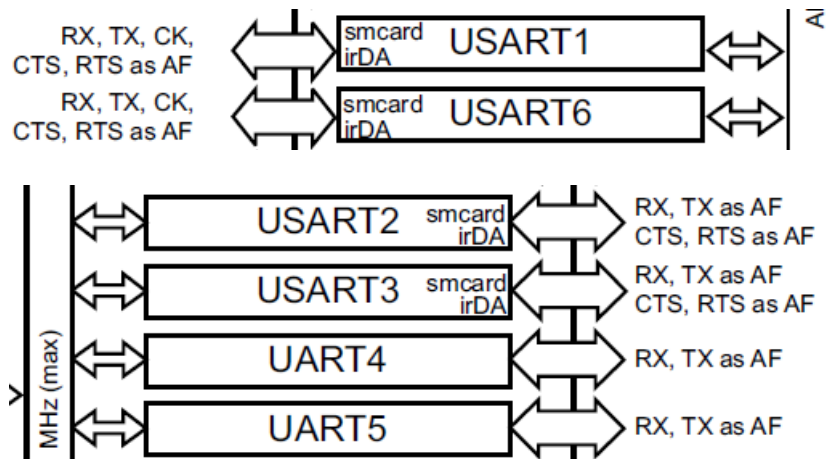


Рис.42. Порти UART мікроконтролера STM32F407VG

Порти USART працюють в режимах синхронної та асинхронної передачі, порти UART – тільки в режимах асинхронної передачі, див. таблицю **Table 147. USART mode configuration** у файлі *RM0090 STM32F405xx_07xx Reference manual.pdf*.

USART mode configuration

Table 147. USART mode configuration⁽¹⁾

USART modes	USART 1	USART 2	USART 3	UART4	UART5	USART 6
Asynchronous mode	X	X	X	X	X	X
Hardware flow control	X	X	X	NA	NA	X
Multibuffer communication (DMA)	X	X	X	X	X	X
Multiprocessor communication	X	X	X	X	X	X
Synchronous	X	X	X	NA	NA	X
Smartcard	X	X	X	NA	NA	X
Half-duplex (single-wire mode)	X	X	X	X	X	X
IrDA	X	X	X	X	X	X
LIN	X	X	X	X	X	X

1. X = supported; NA = not applicable.

Рис.43. Режими конфігурації портів UART мікроконтролера STM32F407VG

30.6 USART registers

Refer to [Section 1.1 on page 57](#) for a list of abbreviations used in register descriptions.

The peripheral registers have to be accessed by half-words (16 bits) or words (32 bits).

30.6.1 Status register (USART_SR)

Address offset: 0x00

Reset value: 0x00C0 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NF	FE	PE
						rc_w0	rc_w0	r	rc_w0	rc_w0	r	r	r	r	r

30.6.3 Baud rate register (USART_BRR)

Note: The baud counters stop counting if the TE or RE bits are disabled respectively.

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIV_Mantissa[11:0]												DIV_Fraction[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

30.6.4 Control register 1 (USART_CR1)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw	Res.	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Рис.43. Регістри портів UART мікроконтролера STM32F407VG

Детальніше про регістри портів UART та їх структуру див. у файлі *RM0090 STM32F405xx_07xx Reference manual.pdf*.

В лабораторній роботі для передачі значень відліків АЦП в комп'ютер використовується порт USART2. Ініціалізація порту наведена на Рис.44.

```

/* Definition for USARTx clock resources */
#define USARTx ..... USART2
#define USARTx_CLK_ENABLE() ..... __HAL_RCC_USART2_CLK_ENABLE();
#define USARTx_RX_GPIO_CLK_ENABLE() ..... __HAL_RCC_GPIOA_CLK_ENABLE()
#define USARTx_TX_GPIO_CLK_ENABLE() ..... __HAL_RCC_GPIOA_CLK_ENABLE()

#define USARTx_FORCE_RESET() ..... __HAL_RCC_USART2_FORCE_RESET()
#define USARTx_RELEASE_RESET() ..... __HAL_RCC_USART2_RELEASE_RESET()

/* Definition for USARTx Pins */
#define USARTx_TX_PIN ..... GPIO_PIN_2
#define USARTx_TX_GPIO_PORT ..... GPIOA
#define USARTx_TX_AF ..... GPIO_AF7_USART2
#define USARTx_RX_PIN ..... GPIO_PIN_3
#define USARTx_RX_GPIO_PORT ..... GPIOA
#define USARTx_RX_AF ..... GPIO_AF7_USART2

```

```

UartHandle.Instance ..... = USARTx;
.
.
UartHandle.Init.BaudRate ..... = 9600;
UartHandle.Init.WordLength ..... = UART_WORDLENGTH_8B;
UartHandle.Init.StopBits ..... = UART_STOPBITS_1;
UartHandle.Init.Parity ..... = UART_PARITY_NONE;
UartHandle.Init.HwFlowCtl ..... = UART_HWCONTROL_NONE;
UartHandle.Init.Mode ..... = UART_MODE_TX_RX;
UartHandle.Init.OverSampling = UART_OVERSAMPLING_16;
. . .
if(HAL_UART_Init(&UartHandle) != HAL_OK)
{
    Error_Handler();
}

```

Рис.44. Ініціалізація порту USART2

```

if (HAL_UART_Transmit(&UartHandle, (uint8_t*)aTxBuffer, TXBUFFERSIZE, 5000) != HAL_OK)
{
    Error_Handler();
}

```

Рис.45. Передача байту через порт USART2

Порт UART (USART2) підключається до USB комп'ютера через перетворювач UART-USB типу CP2102, див. файл *CP2102-9.pdf*.

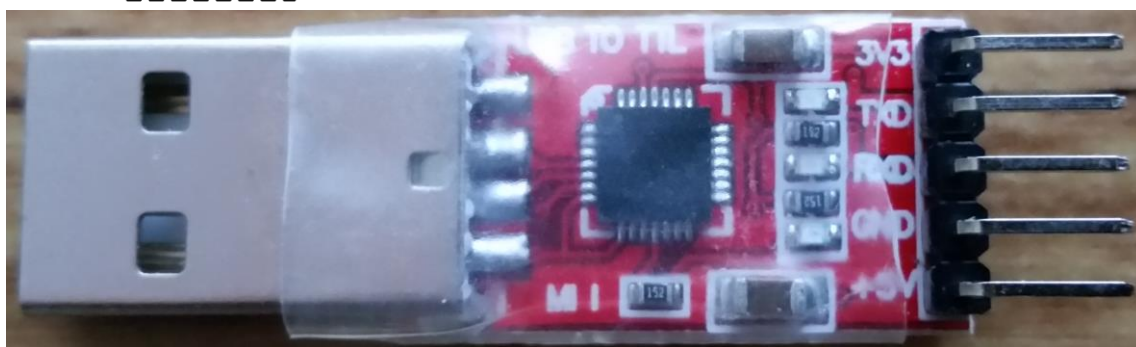
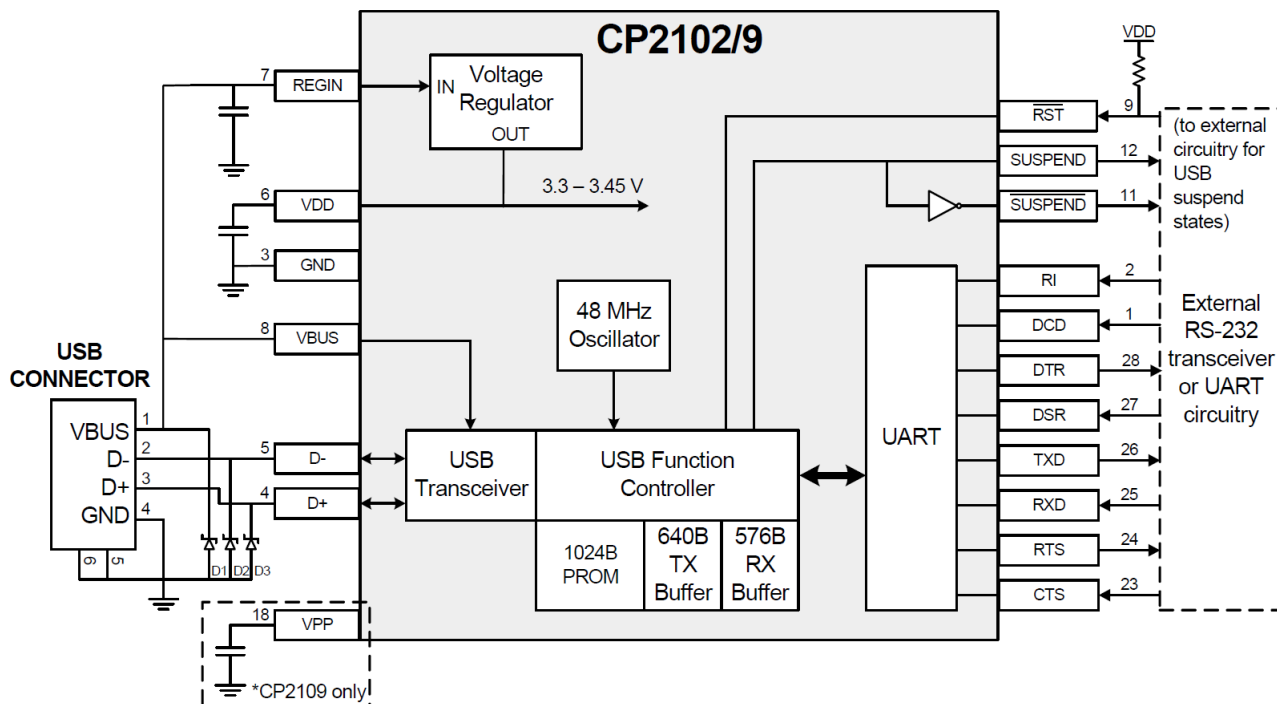


Рис.46. Внутрішня структура та зовнішній вигляд модуля перетворювача UART-USB типу CP2102

PA2	USART2_TX/ TIM5_CH3/ TIM9_CH1/ TIM2_CH3/ ETH_MDIO/ ADC123_IN2
PA3	USART2_RX/ TIM5_CH4/ TIM9_CH2/ TIM2_CH4/ OTG_HS_ULPI_D0/ ETH_MII_COL/ ADC123_IN3

Рис.47. Лінії TXD (порт PA2), RXD (порт PA3) USART2

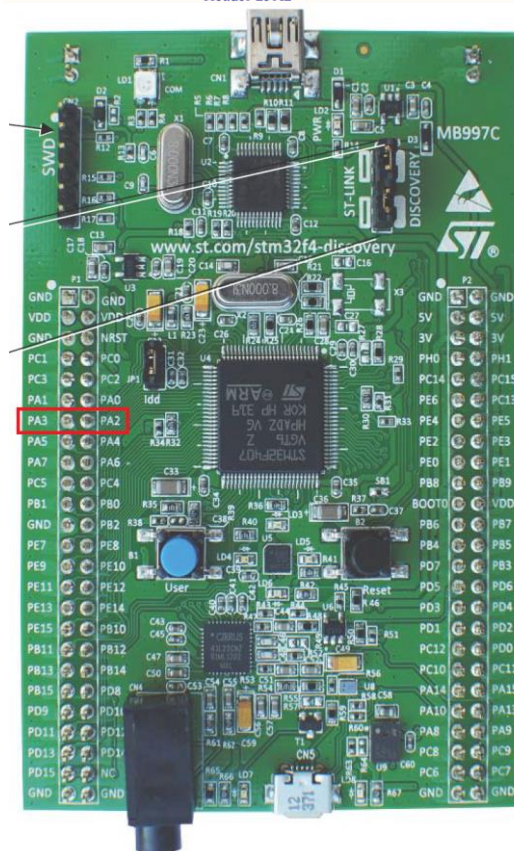
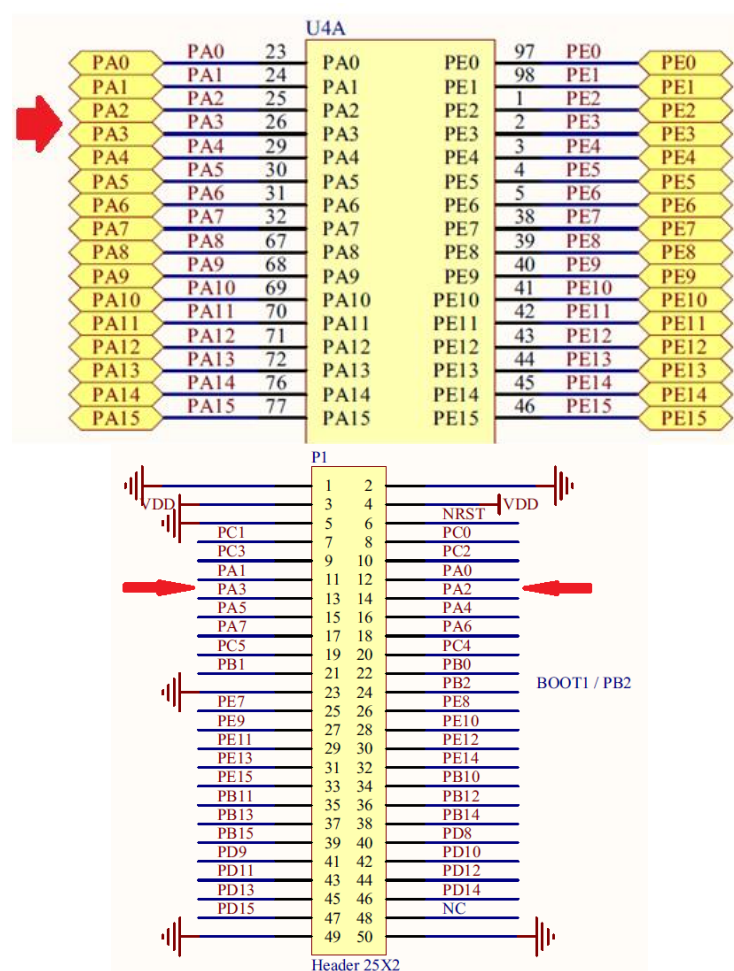


Рис.48. Контакти підключення до ліній TXD (порт PA2), RXD (порт PA3) USART2 на платі STM32F4DISCOVERY

Література

1. RM0090 STM32F405xx_07xx Reference manual.pdf
2. STM32F407xx.pdf
3. STM32F4DISCOVERY User manual.pdf
4. hd44780u.pdf.
5. CP2102-9.pdf
6. Конспект лекцій з дисципліни «Мікропроцесорні системи»

Лістинг програмного модуля перетворення АЦП, виводу результату на екран і передачі по UART

```
/* Includes -----*/
#include "main.h"
#include <stdio.h>
#include <stm32f4xx_hal_uart.h>

/* ADC handler declaration */
ADC_HandleTypeDef AdcHandle;

/* Variable used to get converted value */
__IO uint16_t uhADCxConvertedValue = 0;

/* Private function prototypes -----*/
static void SystemClock_Config(void);
static void Error_Handler(void);

/* Private functions -----*/
//=====
//LCD1602
#define LCM_OUT GPIOB->ODR

#define LCM_PIN_RS GPIO_PIN_8 // PB8
#define LCM_PIN_EN GPIO_PIN_1 // PB1
#define LCM_PIN_D7 GPIO_PIN_7 // PB7
#define LCM_PIN_D6 GPIO_PIN_6 // PB6
#define LCM_PIN_D5 GPIO_PIN_5 // PB5
#define LCM_PIN_D4 GPIO_PIN_4 // PB4

#define LCM_PIN_MASK ((LCM_PIN_RS | LCM_PIN_EN | LCM_PIN_D7 | LCM_PIN_D6 | \
LCM_PIN_D5 | LCM_PIN_D4))
GPIO_InitTypeDef GPIO_InitStructure;

void delay(int a)
{
    int i = 0;
    int f = 0;
    while (f < a)
    {
        while (i < 60)
        {i++; }
        f++;
    }
}

void PulseLCD()
{
    LCM_OUT &= ~LCM_PIN_EN;
    delay(10000);
    LCM_OUT |= LCM_PIN_EN;
    delay(10000);
    LCM_OUT &= (~LCM_PIN_EN);
    delay(10000);
}
```

```

}

//---Запис байта в дисплей---//
void SendByte(char ByteToSend, int IsData)
{
    LCM_OUT &= (~LCM_PIN_MASK);
    LCM_OUT |= (ByteToSend & 0xF0);

    if (IsData == 1)
        LCM_OUT |= LCM_PIN_RS;
    else
        LCM_OUT &= ~LCM_PIN_RS;
    PulseLCD();
    LCM_OUT &= (~LCM_PIN_MASK);
    LCM_OUT |= ((ByteToSend & 0x0F) << 4);

    if (IsData == 1)
        LCM_OUT |= LCM_PIN_RS;
    else
        LCM_OUT &= ~LCM_PIN_RS;

    PulseLCD();
}

//---Установка позиції курсора---//
void Cursor(char Row, char Col)
{
    char address;
    if (Row == 0)
        address = 0;
    else
        address = 0x40;
    address |= Col;
    SendByte(0x80 | address, 0);
}

//---Очистка дисплея---//
void ClearLCDScreen()
{
    SendByte(0x01, 0);
    SendByte(0x02, 0);
}

//---Ініціалізація дисплея---//

static void InitLCD(void);
static void InitLCD(void)
{
    __GPIOB_CLK_ENABLE();

    GPIO_InitTypeDef GPIO_InitStructure;

    GPIO_InitStructure.Pin = GPIO_PIN_1 | GPIO_PIN_4 | GPIO_PIN_5 |

```

```

        GPIO_PIN_6 | GPIO_PIN_7 | GPIO_PIN_8;
    GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_HIGH;
    GPIO_InitStructure.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStructure);

    LCM_OUT &= ~(LCM_PIN_MASK);
    delay(10000);

    LCM_OUT &= ~LCM_PIN_RS;
    LCM_OUT &= ~LCM_PIN_EN;
    LCM_OUT = 0x20;
    PulseLCD();
    SendByte(0x28, 0);
    SendByte(0x0E, 0);
    SendByte(0x06, 0);
}
//---Вивід рядка---//
void PrintStr(char *Text)
{
    char *c;
    c = Text;
    while ((c != 0) && (*c != 0))
    {
        SendByte(*c, 1);
        c++;
    }
}

char str[17];
float t;
//=====================================================
//UART
uint8_t aTxBuffer[] = { 0x01 };
UART_HandleTypeDef UartHandle;

/* Size of Transmission buffer */
#define TXBUFFERSIZE      (COUNTOF(aTxBuffer) - 1)
#define TXBUFFERSIZE      (COUNTOF(aTxBuffer))
/* Size of Reception buffer */
#define RXBUFFERSIZE      TXBUFFERSIZE

/* Exported macro -----*/
#define COUNTOF(__BUFFER__) (sizeof(__BUFFER__) / sizeof(*(__BUFFER__)))

/* User can use this section to tailor USARTx/UARTx instance used and associated
resources */
/* Definition for USARTx clock resources */
#define USARTx             USART2
#define USARTx_CLK_ENABLE() __HAL_RCC_USART2_CLK_ENABLE();
#define USARTx_RX_GPIO_CLK_ENABLE() __HAL_RCC_GPIOA_CLK_ENABLE()
#define USARTx_TX_GPIO_CLK_ENABLE() __HAL_RCC_GPIOA_CLK_ENABLE()

#define USARTx_FORCE_RESET() __HAL_RCC_USART2_FORCE_RESET()

```

```

#define USARTx_RELEASE_RESET()      __HAL_RCC_USART2_RELEASE_RESET()

/* Definition for USARTx Pins */
#define USARTx_TX_PIN                GPIO_PIN_2
#define USARTx_TX_GPIO_PORT          GPIOA
#define USARTx_TX_AF                  GPIO_AF7_USART2
#define USARTx_RX_PIN                GPIO_PIN_3
#define USARTx_RX_GPIO_PORT          GPIOA
#define USARTx_RX_AF                  GPIO_AF7_USART2

/*###-1- Configure the UART peripheral #####*/
/* Put the USART peripheral in the Asynchronous mode (UART Mode) */
/* UART1 configured as follow:
   - Word Length = 8 Bits
   - Stop Bit = One Stop bit
   - Parity = None
   - BaudRate = 9600 baud
   - Hardware flow control disabled (RTS and CTS signals) */

void InitUART(void)
{
    UartHandle.Instance              = USARTx;

    UartHandle.Init.BaudRate         = 9600;
    UartHandle.Init.WordLength       = UART_WORDLENGTH_8B;
    UartHandle.Init.StopBits         = UART_STOPBITS_1;
    UartHandle.Init.Parity           = UART_PARITY_NONE;
    UartHandle.Init.HwFlowCtl        = UART_HWCONTROL_NONE;
    UartHandle.Init.Mode              = UART_MODE_TX_RX;
    UartHandle.Init.OverSampling     = UART_OVERSAMPLING_16;

    if (HAL_UART_Init(&UartHandle) != HAL_OK)
    {
        Error_Handler();
    }
}

//=====================================================
int main(void)
{
    ADC_ChannelConfTypeDef sConfig;

    /* STM32F4xx HAL library initialization:
       - Configure the Flash prefetch, instruction and Data caches
       - Configure the SysTick to generate an interrupt each 1 msec
       - Set NVIC Group Priority to 4
       - Global MSP (MCU Support Package) initialization
    */
    HAL_Init();

    /* Configure the system clock to 144 MHz */
    SystemClock_Config();

```



```

    InitLCD();                //Ініціалізація дисплея
delay(500);

    InitUART(); //Ініціалізація UART

/* Configure LED4 and LED5 */
    //BSP_LED_Init(LED4);
    //BSP_LED_Init(LED5);

    /###-1- Configure the ADC peripheral #####*/
    AdcHandle.Instance = ADCx;

    AdcHandle.Init.ClockPrescaler = ADC_CLOCKPRESCALER_PCLK_DIV2;
    AdcHandle.Init.Resolution = ADC_RESOLUTION_12B;
    AdcHandle.Init.ScanConvMode = DISABLE;
    AdcHandle.Init.ContinuousConvMode = ENABLE;
    AdcHandle.Init.DiscontinuousConvMode = DISABLE;
    AdcHandle.Init.NbrOfDiscConversion = 0;
    AdcHandle.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
    AdcHandle.Init.ExternalTrigConv = ADC_EXTERNALTRIGCONV_T1_CC1;
    AdcHandle.Init.DataAlign = ADC_DATAALIGN_RIGHT;
    AdcHandle.Init.NbrOfConversion = 1;
    // AdcHandle.Init.DMAContinuousRequests = ENABLE;
    AdcHandle.Init.DMAContinuousRequests = DISABLE;
    AdcHandle.Init.EOCSelection = DISABLE;

    if (HAL_ADC_Init(&AdcHandle) != HAL_OK)
    {
        /* Initialization Error */
        Error_Handler();
    }

    /###-2- Configure ADC regular channel #####*/
    /* Note: Considering IT occurring after each number of size of      */
    /* "uhADCxConvertedValue" ADC conversions (IT by DMA end          */
    /* of transfer), select sampling time and ADC clock with sufficient */
    /* duration to not create an overhead situation in IRQHandler.    */
    sConfig.Channel = ADCx_CHANNEL;
    sConfig.Rank = 1;
    sConfig.SamplingTime = ADC_SAMPLETIME_28CYCLES;
    sConfig.Offset = 0;

    if (HAL_ADC_ConfigChannel(&AdcHandle, &sConfig) != HAL_OK)
    {
        /* Channel Configuration Error */
        Error_Handler();
    }

    ClearLCDScreen();        //Очистка дисплея
delay(500);
    Cursor(0, 2);           //Установка курсора в рядку 1
delay(500);

```

```

        PrintStr(" Test LCD");    //Вивід тексту
delay(500);
        Cursor(1, 4);            //Установка курсора в рядку 2
delay(500);
        PrintStr("BC1602AG");
        SendByte(0b00001100, 0); //Курсор виключено

/* Infinite loop */
while(1)
{
    a = 0;

    if (HAL_ADC_Start_DMA(&AdcHandle, (uint32_t*)&uhADCxConvertedValue, 1)
    != HAL_OK)
    {
        /* Start Conversation Error */
        Error_Handler();
    }

    Cursor(0, 1);                //Установка курсора в рядку 1
    delay(500);
    PrintStr("0.1 Meas Ua, B");  //Вивід тексту
    delay(500);
    Cursor(1, 4);                //Установка курсора в рядку 2
    delay(500);

    snprintf(str, 17, " %d      ", uhADCxConvertedValue);
    PrintStr(str);

    if (HAL_ADC_Stop_DMA(&AdcHandle) != HAL_OK)
    {
        /* Start Conversation Error */
        Error_Handler();
    }

    aTxBuffer[0] = (uint8_t)(uhADCxConvertedValue & 0x00ff);

    if (HAL_UART_Transmit(&UartHandle, (uint8_t*)aTxBuffer, TXBUFFERSIZE,
5000) != HAL_OK)
    {
        Error_Handler();
    }

    aTxBuffer[0] = (uint8_t)(uhADCxConvertedValue & 0xff00);

    if (HAL_UART_Transmit(&UartHandle, (uint8_t*)aTxBuffer, TXBUFFERSIZE,
5000) != HAL_OK)
    {
        Error_Handler();
    }
}
}

```

```

/**
 * @brief System Clock Configuration
 * The system Clock is configured as follow :
 * System Clock source      = PLL (HSE)
 * SYSCLK(Hz)               = 144000000
 * HCLK(Hz)                 = 144000000
 * AHB Prescaler            = 1
 * APB1 Prescaler           = 4
 * APB2 Prescaler           = 2
 * HSE Frequency(Hz)       = 8000000
 * PLL_M                    = 8
 * PLL_N                    = 288
 * PLL_P                    = 2
 * PLL_Q                    = 6
 * VDD(V)                   = 3.3
 * Main regulator output voltage = Scale2 mode
 * Flash Latency(WS)       = 4
 * @param None
 * @retval None
 */
static void SystemClock_Config(void)
{
    RCC_ClkInitTypeDef RCC_ClkInitStruct;
    RCC_OscInitTypeDef RCC_OscInitStruct;

    /* Enable Power Control clock */
    __HAL_RCC_PWR_CLK_ENABLE();

    /* The voltage scaling allows optimizing the power consumption when the device is
       clocked below the maximum system frequency, to update the voltage scaling value
       regarding system frequency refer to product datasheet. */
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE
2);

    /* Enable HSE Oscillator and activate PLL with HSE as source */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLM = 8;
    RCC_OscInitStruct.PLL.PLLN = 288;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
    RCC_OscInitStruct.PLL.PLLQ = 6;
    HAL_RCC_OscConfig(&RCC_OscInitStruct);

    /* Select PLL as system clock source and configure the HCLK, PCLK1 and PCLK2
       clocks dividers */
    RCC_ClkInitStruct.ClockType = (RCC_CLOCKTYPE_SYSCLK |
RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2);
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;

```

```

    HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4);

    /* STM32F405x/407x/415x/417x Revision Z devices: prefetch is supported */
    if (HAL_GetREVID() == 0x1001)
    {
        /* Enable the Flash prefetch */
        __HAL_FLASH_PREFETCH_BUFFER_ENABLE();
    }
}

/**
 * @brief This function is executed in case of error occurrence.
 * @param None
 * @retval None
 */
static void Error_Handler(void)
{
    /* Turn LED5 on */
    BSP_LED_On(LED5);
    while (1)
    {
    }
}

/**
 * @brief Conversion complete callback in non blocking mode
 * @param AdcHandle : AdcHandle handle
 * @note This example shows a simple way to report end of conversion, and
 * you can add your own implementation.
 * @retval None
 */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* AdcHandle)
{
    /* Turn LED4 on: Transfer process is correct */
    BSP_LED_On(LED4);
}

/***** (C) COPYRIGHT STMicroelectronics *****/

```

ДОСЛІДЖЕННЯ ІНТЕГРОВАНОГО НА КРИСТАЛІ МІКРОКОНТРОЛЕРА АНАЛОГО-ЦИФРОВОГО ПЕРЕТВОРЮВАЧА

Методичні вказівки

до лабораторної роботи № 1 з курсу “ Мікропроцесорні системи ”
для студентів спеціальності 6.123.00.00 - “Комп’ютерна інженерія”

Укладач

Пуйда В. Я., к. т. н, доцент

Редактор

Комп’ютерне верстання

Здано у видавництво . Підписано до друку
Формат 70х100/16. Папір офсетний. Друк на різнографі
Умовн. друк. арк. Обл.-вид. арк..
Тираж прим. Зам..

Видавництво Національного університету “Львівська політехніка”
Реєстраційне свідоцтво ДК №751 від 27.12.2001 р.

Поліграфічний центр Видавництва
Національного університету “Львівська політехніка”

Вул.. Ф. Колесси, 2. Львів, 79000